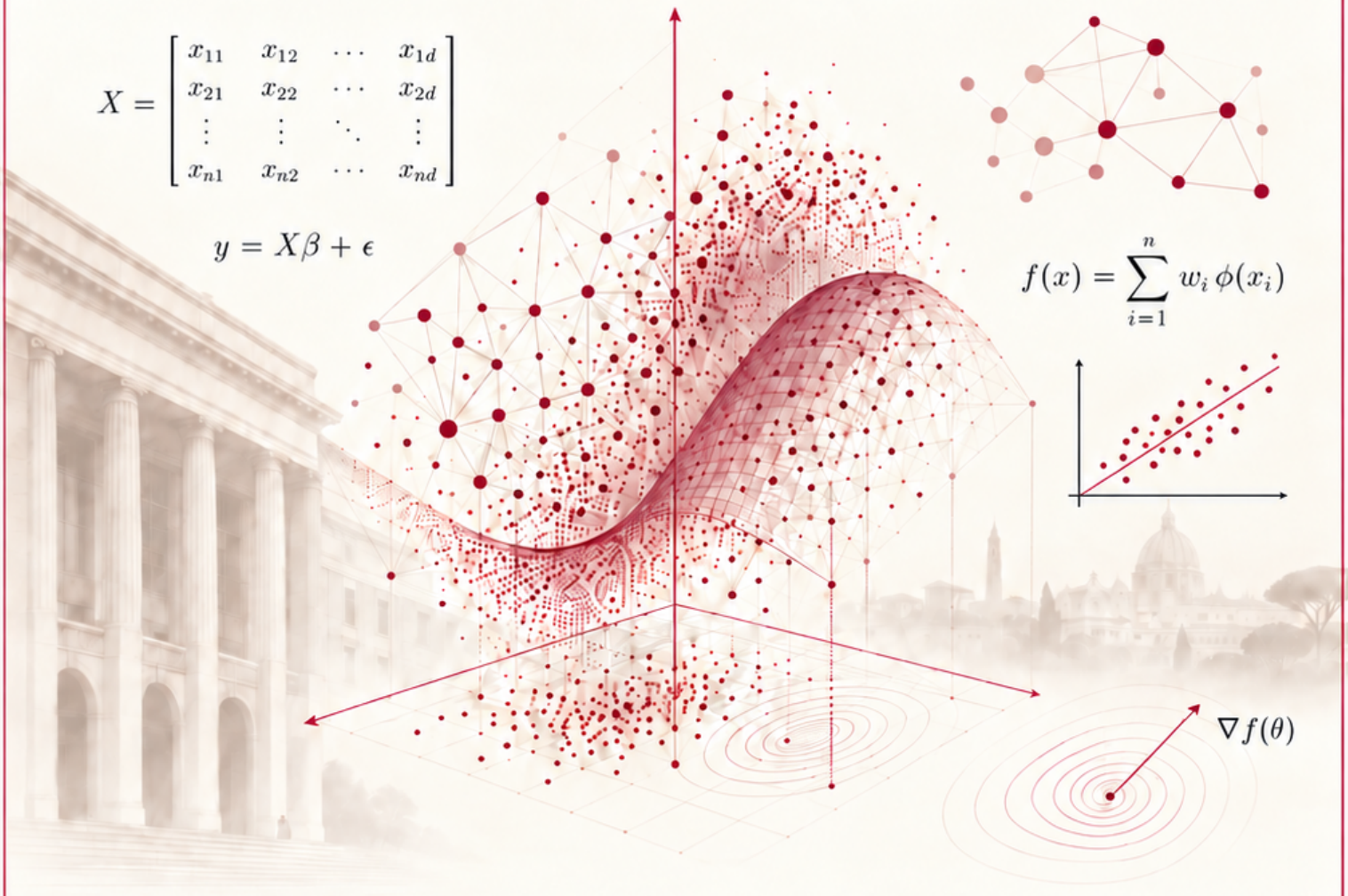
# FOUNDATIONS OF DATA SCIENCE

## Comprehensive Study Notes





Department of Computer Science

NOTES PREPARED BY

Matt Clifford Trinidad Ramos

# Contents


**I Data Science and Machine Learning Foundations** **1**

**1 What Is Data Science?** **1**

**2 Machine Learning as a Subfield of AI** **2**

2.1 Tom Mitchell's learning-task formulation . . . 2

2.2 Why machine learning? . . . 2

2.3 Machine learning and statistics . . . 3

**3 Learning Paradigms** **3**

**4 Supervised Learning** **3**

**5 Representative Supervised-Learning Tasks** **3**

5.1 Regression . . . 4

5.2 Classification . . . 4

5.3 A supervised model as a family of equations . . . 6

5.4 Structured and high-dimensional outputs . . . 6

5.5 Sequence and cross-modal generation . . . 7

**6 Unsupervised Learning and Generative Models** **8**

6.1 Latent variables . . . 9

**7 Reinforcement Learning** **10**

7.1 Chess as an example . . . 10

7.2 Why reinforcement learning is difficult . . . 10

**8 Deep-Learning Landmarks** **11**

**II Data Representation and Linear Algebra Foundations** **12**

**9 The Iris Dataset and Supervised-Learning Data** **12**

9.1 Features and target . . . 12

9.2 Data types and numerical encoding . . . 12

9.3 Labels and dataset notation . . . 13

**10 Data Generation and the i.i.d. Assumption** **13**

**11 Data Points as Vectors** **13**

**12 Spanning Sets and Bases** **14**

12.1 Spanning sets . . . 14

12.2 Basis . . . 14

**13 Standard and Orthonormal Bases** **15**

13.1 The standard basis . . . 15

13.2 Encoding with an orthonormal basis . . . 16

**14 Non-Standard Bases and Coordinate Transformations** **16**

**15 Finding the Encoding Weights** **17**

**16 Subspaces and Projection** **18**

16.1 Encoding, decoding, and projection . . . 19

**III Optimization and Automatic Differentiation** **21**

**17 Optimization Fundamentals** **21**

17.1 Global and local optima . . . 21

17.2 Zero-order and first-order conditions . . . 21

**18 Local Linear Approximation and the Gradient** **22**

18.1 Directions of steepest ascent and descent . . . 23
**19 Computing Derivatives 24**
19.1 Computation graphs . . . 24
19.2 Forward-mode automatic differentiation . . . 25
19.3 Reverse mode and backpropagation . . . 25
**20 Gradient Descent 26**
20.1 Convex and nonconvex objectives . . . 27
20.2 Choosing the step size . . . 27
20.3 Cost-function history plots . . . 29
20.4 Stopping criteria . . . 29
20.5 Weaknesses of vanilla gradient descent . . . 29
20.6 Constrained gradient updates for PCA . . . 31

**IV Principal Component Analysis and Dimensionality Reduction 33**
**21 Linear Transformations and Projection 33**
**22 From a Fixed Basis to a Learned Basis 33**
22.1 Reconstruction error and variance . . . 34
**23 Covariance and the First Principal Component 34**
**24 Eigenvalues, Eigenvectors, and Principal Components 35**
**25 Dimensionality Reduction and Unsupervised Representation Learning 36**
25.1 A compact PCA exercise . . . 36
**26 Linear Structure, Manifolds, and Charts 37**
**27 Quick Review and Terminology 38**
**References 38**

Part I

# Data Science and Machine Learning Foundations

## 1 What Is Data Science?

**Definition**

**Data science** is an interdisciplinary field concerned with extracting actionable insight and knowledge from structured and unstructured data, using methods from statistics, mathematics, computer science, and domain expertise.

**Prerequisites.** Calculus and linear algebra (including derivatives and matrix–vector products), and basic probability and statistics (including Gaussian distributions, mean, and standard deviation).

Data science is broader than model fitting. A typical lifecycle starts from the problem that needs to be solved, passes through data and model development, and continues after deployment because models must be monitored and may need retraining.

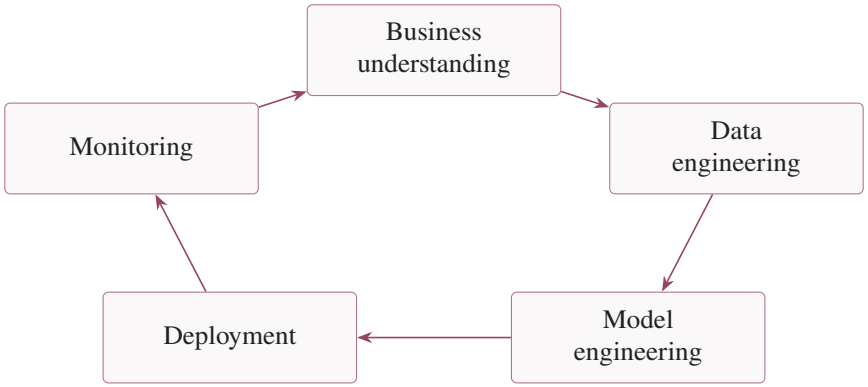



**Figure 1.** Data science lifecycle from business understanding through data engineering, model engineering, deployment, and monitoring. Feedback from monitoring initiates a new development cycle.

The stages involve the following activities:

- **Business understanding:** define business objectives and translate them into machine-learning objectives.
- **Data engineering:** collect and verify data, clean and visualize it, augment it when appropriate, and standardize it.
- **Model engineering:** select a model, train it, and evaluate it.
- **Deployment:** move the model into production and consider user acceptance and usability.
- **Monitoring:** monitor efficacy, collect new data, and retrain when required.

## 2 Machine Learning as a Subfield of AI

Artificial Intelligence (AI) is the field of computer science devoted to creating agents capable of tasks that typically require human intelligence, such as learning, reasoning, and problem solving. Supervised learning, unsupervised learning, and reinforcement learning are major machine-learning paradigms within AI. Deep learning cuts across these paradigms.

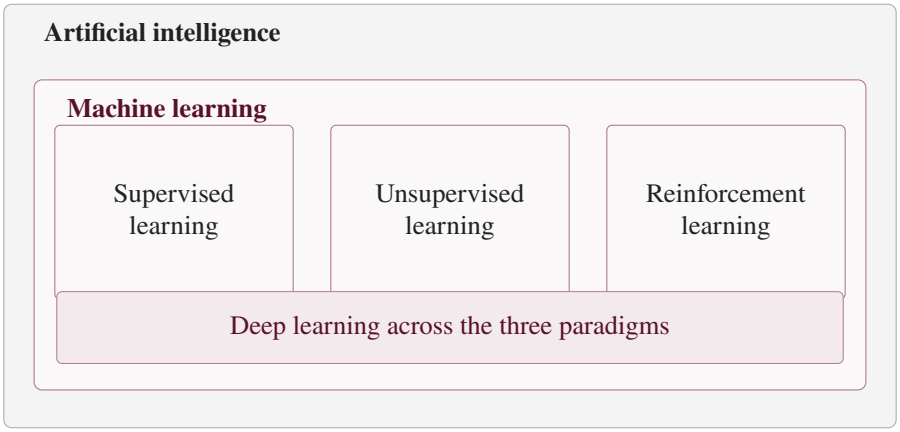



**Figure 2.** Artificial intelligence, machine learning, and the three learning paradigms. Deep learning supplies model families across the paradigms; the diagram is conceptual rather than a strict set-theoretic taxonomy.

### 2.1 Tom Mitchell's learning-task formulation

In Tom Mitchell's learning-task formulation, a machine-learning system improves its **performance** $P$ at a **task** $T$ through **experience** $E$. A well-defined learning problem can therefore be summarized by the triple

$$\langle P, T, E \rangle.$$

**Notation.** $T$ denotes the task to be performed; $P$ is the quantitative performance measure; $E$ is the experience or data from which the system learns.

**Table 1.** Examples of tasks, performance measures, and learning experience.

| **Task** $T$ | **Performance** $P$ | **Experience** $E$ |
|---|---|---|
| Playing checkers | Percentage of games won against an arbitrary opponent | Practice games played against itself |
| Recognizing handwritten words | Percentage of words correctly classified | Human-labeled images of handwritten words |
| Driving on four-lane highways using vision sensors | Average distance traveled before a human-judged error | Images and steering commands recorded while observing a human driver |
| Classifying email as spam or legitimate | Percentage of messages correctly classified | A database of emails with human-provided labels |

### 2.2 Why machine learning?

Many useful tasks are too complex to specify with explicit hand-written rules. Object recognition, speech understanding, and natural-language processing are canonical examples. Instead of manually encoding all rules, machine-learning systems learn useful regularities automatically from data or experience.

### 2.3 Machine learning and statistics

Statistics and machine learning share substantial mathematical and methodological foundations: both fields uncover patterns in data, rely on calculus, probability, and linear algebra, and share many algorithms. Their differences lie mainly in emphasis: statistics traditionally focuses more strongly on inference, conclusions, interpretability, and rigor, whereas machine learning often emphasizes prediction, scalability, and autonomy. These differences in goals and emphasis do not establish a hard boundary between the disciplines.

## 3 Learning Paradigms

**Table 2.** Input/output view of the three paradigms.

| Paradigm | Input | Output |
|---|---|---|
| Supervised learning | Examples of inputs paired with desired outputs | A model that predicts an output for a new input |
| Unsupervised learning | Examples of data without target outputs | A representation of structure in the data |
| Reinforcement learning | A sequence of interactions with an environment | A policy that performs a desired task |

## 4 Supervised Learning

Supervised learning defines a mapping from input to output and learns this mapping from paired input/output examples. In traditional programming, *data* + *program* produce the output; in machine learning, *data* + *observed outputs* are used to learn the program/model.

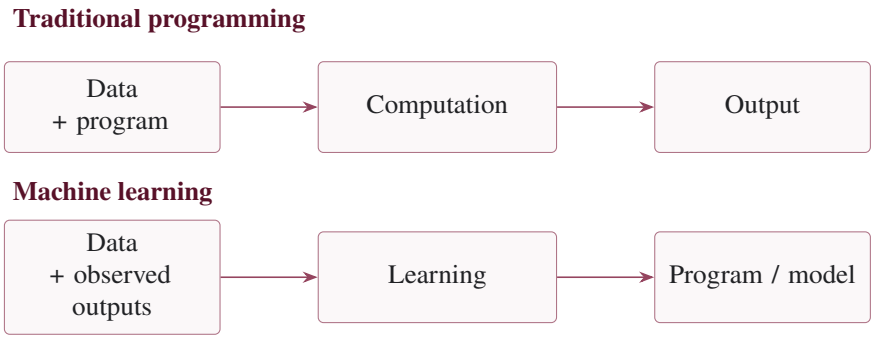



**Figure 3.** Traditional programming and machine learning: a program computes outputs, whereas learning uses observed inputs and outputs to construct a model.

## 5 Representative Supervised-Learning Tasks

A common prediction pipeline encodes raw real-world objects numerically, processes them with a model, and decodes the results into meaningful outputs. The examples differ primarily in the structure and dimensionality of the target.

## 5.1 Regression

Regression predicts real-valued quantities. In the univariate example, real-estate information is encoded as a feature vector and mapped to a single predicted price. In the graph-regression example, a molecular graph is encoded and mapped to more than one continuous target, namely physical properties such as freezing and boiling point.

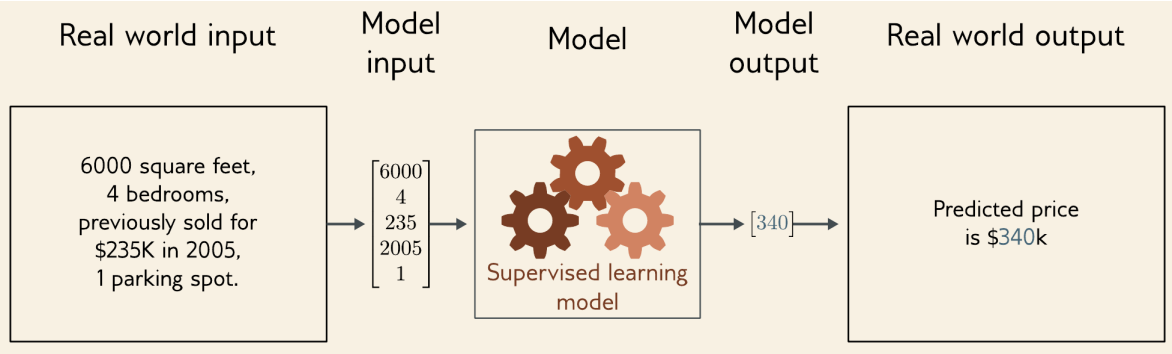



**Figure 4.** Univariate regression example: a real-world description is encoded numerically and mapped to a single real-valued output.

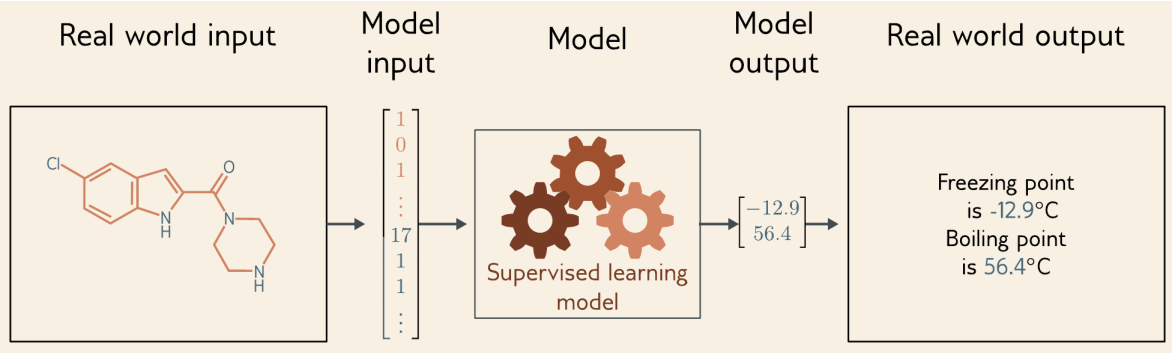



**Figure 5.** Graph regression example: a molecular structure is represented numerically and mapped to multiple continuous outputs.

## 5.2 Classification

Classification predicts discrete classes. A binary text-classification example assigns a sentence to one of two classes (positive/negative) using a Transformer. A music-genre example and an image-classification example illustrate multiclass prediction using, respectively, a recurrent neural network (RNN) and a convolutional network.

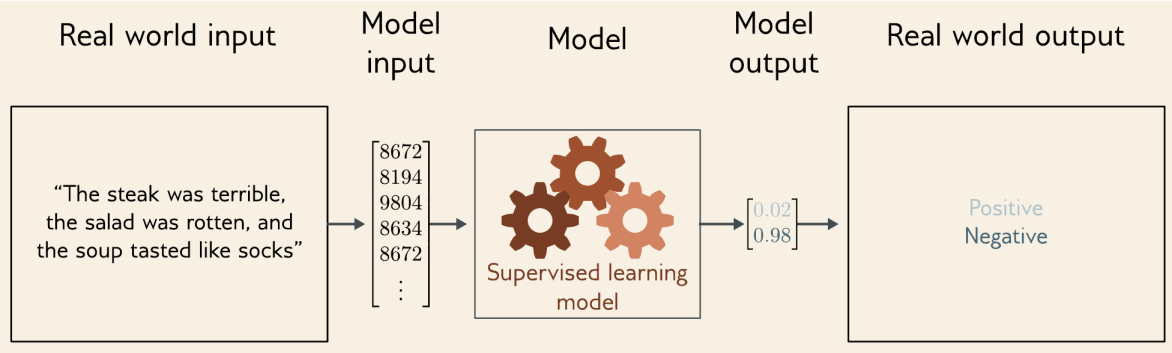



**Figure 6.** Binary text classification: text → numerical representation → class scores → positive/negative label.

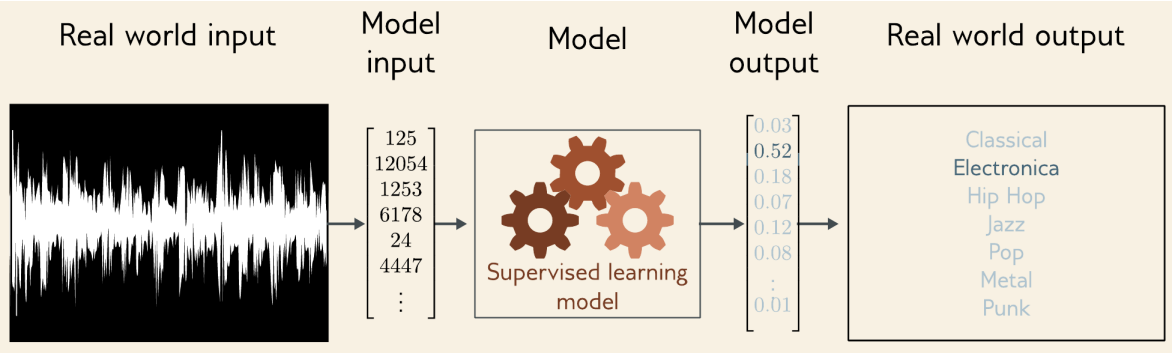



**Figure 7.** Multiclass music-genre classification using a recurrent neural network.

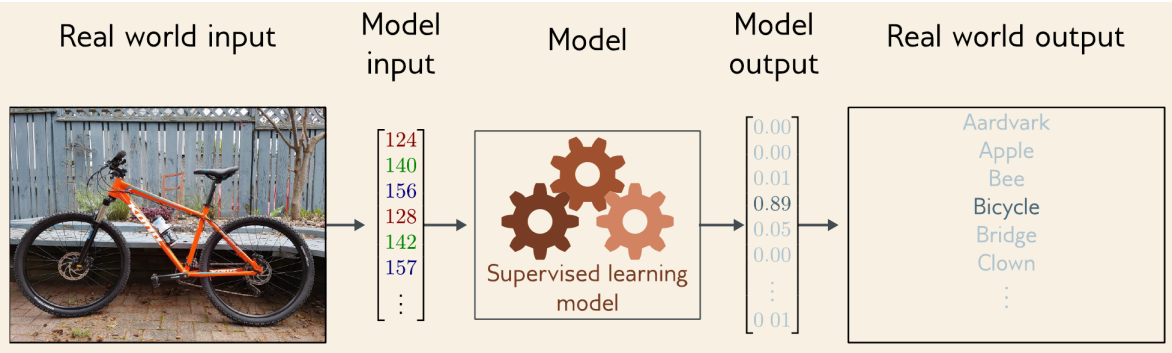



**Figure 8.** Multiclass image classification using a convolutional network.

## 5.3 A supervised model as a family of equations

A supervised model can predict height from age. Training means searching through a family of candidate equations and selecting one that fits the training observations well. Deep neural networks form extremely flexible families of equations; fitting such networks is an instance of **deep learning**.

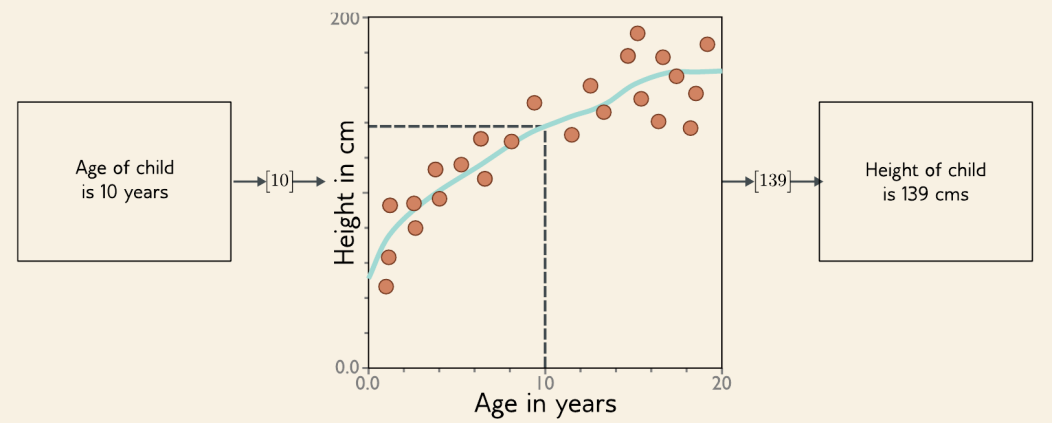



**Figure 9.** Supervised-learning view of model fitting: an input value is mapped to an output through an equation fitted to training data.

## 5.4 Structured and high-dimensional outputs

Some supervised tasks produce many outputs rather than a single scalar. Examples include:

- **Image segmentation:** many binary/discrete outputs, one per spatial location; the example uses a convolutional encoder–decoder network.
- **Depth estimation:** many continuous outputs forming a dense depth map; using a convolutional encoder–decoder.
- **Pose estimation:** many continuous outputs describing body/keypoint geometry; using a convolutional encoder–decoder.

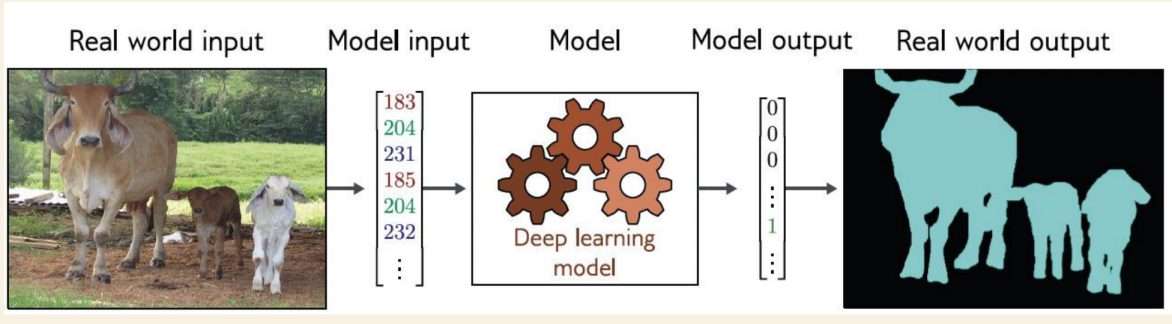



**Figure 10.** Image segmentation example: an input image is mapped to a dense discrete mask.

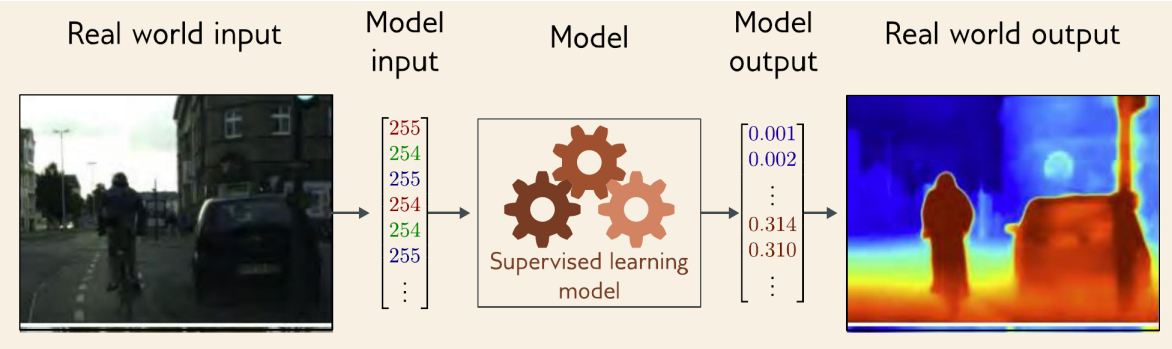



**Figure 11.** Depth estimation example: an input image is mapped to a dense continuous depth representation.

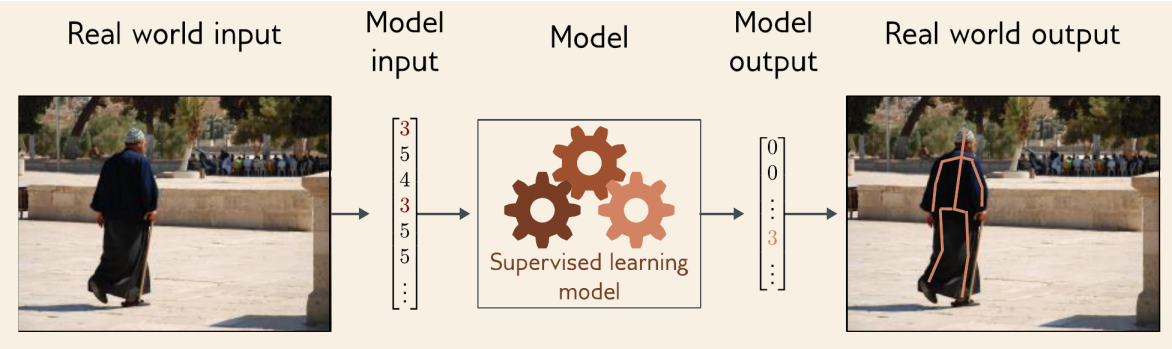



**Figure 12.** Pose estimation example: an input image is mapped to multiple continuous outputs representing pose.

## 5.5 Sequence and cross-modal generation

The same input–model–output pattern extends to structured sequences and multimodal tasks. Examples include machine translation (text sequence to text sequence), image captioning (image to text sequence), and text-to-image generation (text sequence to image).

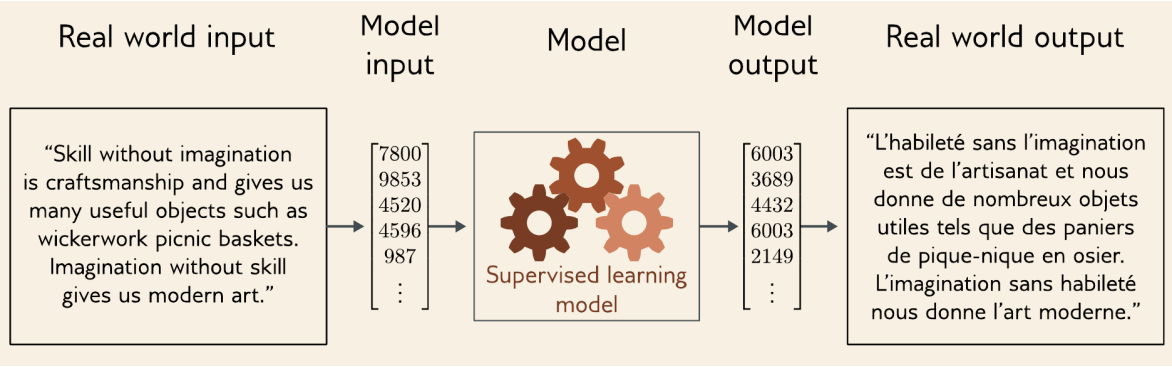



**Figure 13.** Machine-translation example: text is tokenized to a numerical representation and decoded into a target-language sequence.

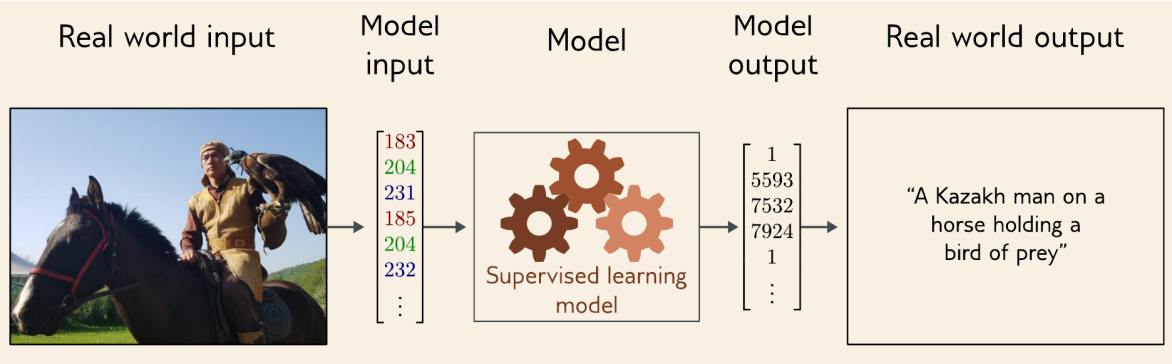



**Figure 14.** Image-captioning example: an image is encoded numerically and decoded into a textual description.

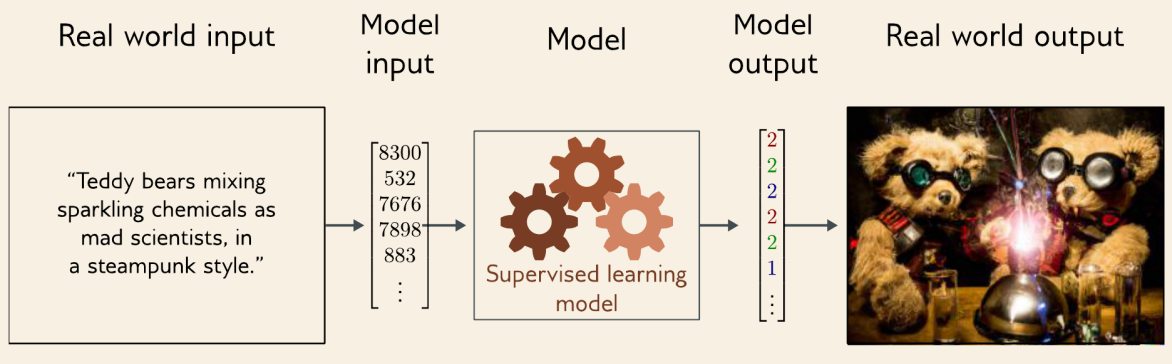



**Figure 15.** Text-to-image example: a text prompt is encoded and transformed into an image.

**Key Concept**

Across these examples, the relationship between input and output can be highly complex and there may be multiple valid outputs. Nevertheless, outputs—and often inputs—possess structure: language follows grammatical rules and natural images obey strong statistical regularities. This observation motivates learning the "grammar" of data from large quantities of unlabeled examples.

## 6 Unsupervised Learning and Generative Models

Unsupervised learning studies datasets without target labels. Representative goals include clustering, outlier detection, generation of new examples, and filling in missing data.

A central idea is to exploit the abundance of unlabeled data to learn structure before or alongside a supervised task. An example is *DeepCluster: Deep Clustering for Unsupervised Learning of Visual Features* (Caron et al., 2018), where visual examples are organized into coherent feature clusters without conventional class labels.

**Generative models** can create new examples. Approaches include generative adversarial networks (GANs), as well as probabilistic generative models such as variational autoencoders (VAEs), normalizing flows, and diffusion models. Generative modeling is not synonymous with unsupervised learning: conditional generative models may also use labels, paired data, or other forms of supervision.

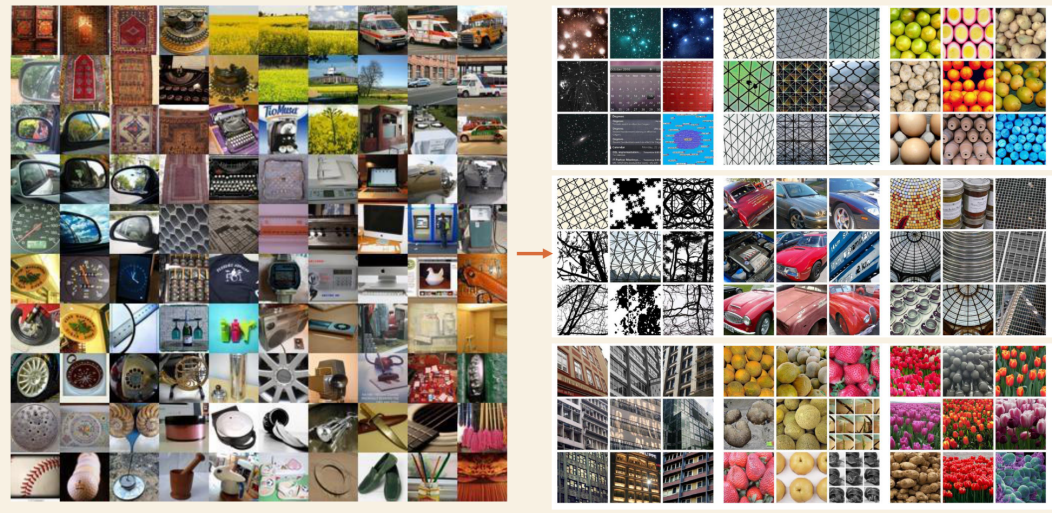


**Figure 16.** DeepCluster: visually coherent groups discovered from unlabeled images.

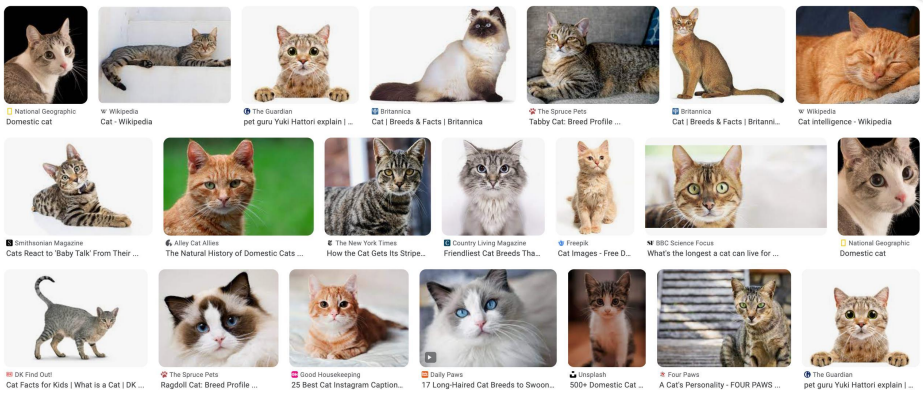


**Figure 17.** Examples of synthetic cat images produced by generative models.

## 6.1 Latent variables

A latent-variable generative model represents high-dimensional observed data through lower-dimensional latent variables. Samples are drawn in the latent space, transformed by the model, and decoded into observations.

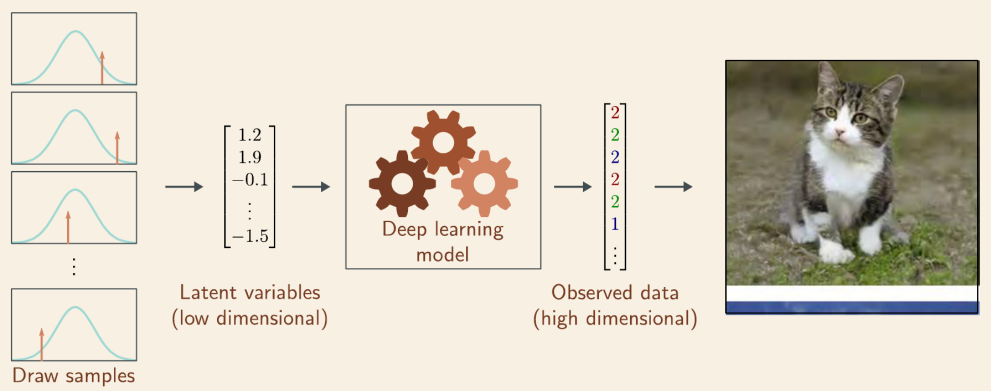



**Figure 18.** Latent-variable generation: low-dimensional latent samples are transformed by a deep model into high-dimensional observed data.

# 7 Reinforcement Learning

Reinforcement learning (RL) involves three basic ingredients: a set of **states**, a set of **actions**, and a set of **rewards**. The agent acts in the environment, changes state, and seeks behavior that produces reward. Unlike ordinary supervised learning, the agent is not given a fixed labeled dataset in advance; it must gather information through interaction.

## 7.1 Chess as an example

In chess, states are valid chess positions and available actions are valid moves. An illustrative reward scheme assigns positive rewards for taking pieces and negative rewards for losing them.

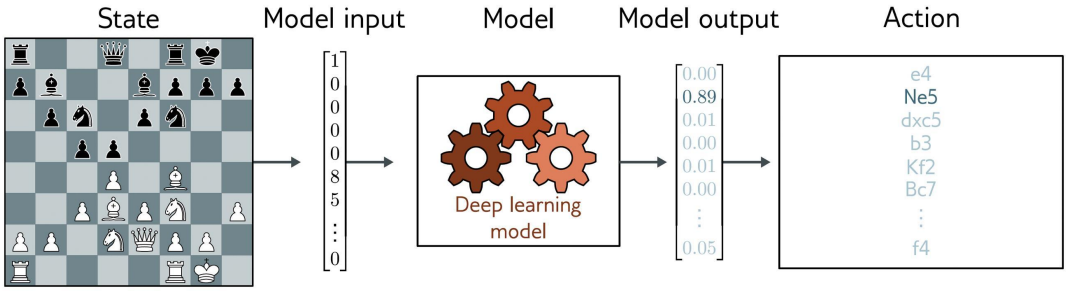



**Figure 19.** Reinforcement-learning pipeline for chess: board state → model representation → action scores → selected move.

> **Remark**
>
> Piece-capture rewards illustrate **reward shaping**: intermediate rewards provide feedback during the state–action–reward interaction loop.

## 7.2 Why reinforcement learning is difficult

Three central difficulties are:

- **Stochasticity:** the same action need not lead to the same future because other agents or the environment may behave differently.
- **Temporal credit assignment:** a reward observed now may be caused by an earlier sequence of decisions rather than the most recent action alone.
- **Exploration–exploitation trade-off:** an agent must decide whether to reuse a known good strategy or explore alternatives that might be better.

## 8 Deep-Learning Landmarks

Yoshua Bengio, Geoffrey Hinton, and Yann LeCun received the 2018 ACM A.M. Turing Award. The following timeline summarizes influential developments in neural networks and deep learning.

**Table 3.** Selected milestones in neural networks and deep learning.

| Year | Milestone |
|---|---|
| 1958 | Perceptron (simple neural model) |
| 1986 | Backpropagation (practical deep neural networks) |
| 1989 | Convolutional networks (supervised learning) |
| 2012 | AlexNet image classification (supervised learning) |
| 2014 | Generative adversarial networks (unsupervised learning) |
| 2014 | Deep Q-learning for Atari games (reinforcement learning) |
| 2016 | AlphaGo (reinforcement learning) |
| 2017 | Machine translation (supervised learning) |
| 2019 | Language models (supervised/unsupervised learning) |
| 2022 | DALL-E 2, image synthesis from text prompts |
| 2022 | ChatGPT |
| 2023 | GPT-4 multimodal model |
| 2024 | Demis Hassabis and John Jumper: Nobel Prize in Chemistry in connection with AlphaFold-related work; Geoffrey Hinton and John Hopfield: Nobel Prize in Physics |
| 2026 | OpenAI announced an AI-generated proof concerning the Navier–Stokes Millennium problem; formal acceptance remains a separate matter. |

**Historical note (2 October 2026).** OpenAI announced the proof on 8 September 2026; this announcement does not by itself establish a finalized Millennium Prize determination. See OpenAI's announcement and the Clay Mathematics Institute's problem page.

**Machine Learning Foundations — Key Takeaways**

- Data science covers the full lifecycle from problem definition and data preparation to deployment and monitoring.
- Machine learning replaces explicit hand-coded rules with models learned from data or interaction.
- Supervised, unsupervised, and reinforcement learning differ primarily in the information available during learning and the object being learned.
- Deep learning supplies highly flexible model families that can be used across several learning paradigms.
- Modern tasks often involve structured, high-dimensional inputs and outputs rather than simple scalar prediction.

Part II

# Data Representation and Linear Algebra Foundations

## 9 The Iris Dataset and Supervised-Learning Data

Fisher's Iris dataset provides a compact supervised-learning example. It contains **150 flowers**: 50 *setosa*, 50 *versicolor*, and 50 *virginica*. Each flower is described by four measurements in centimeters: sepal length, sepal width, petal length, and petal width. The target is the iris species.




**Figure 20.** The three Iris species: versicolor, virginica, and setosa. The classification task predicts the species from sepal and petal measurements.

### 9.1 Features and target

For $n$ observed objects, the **features** are properties used to describe each object, while the **target** is the output variable or prediction goal. Supervised learning assumes that some relationship exists between features and target and attempts to learn a predictive rule from observed examples.

**Table 4.** Illustrative Iris data: four measurements in centimeters and the species label.

| Sepal L. | Sepal W. | Petal L. | Petal W. | Species |
|---|---|---|---|---|
| 4.3 | 3.0 | 1.1 | 0.1 | setosa |
| 5.0 | 3.3 | 1.4 | 0.2 | setosa |
| 7.7 | 3.8 | 6.7 | 2.2 | virginica |
| 5.5 | 2.5 | 4.0 | 1.3 | versicolor |

### 9.2 Data types and numerical encoding

Features and targets can have different types:

- numerical variables in $\mathbb{R}$;
- integer variables in $\mathbb{Z}$;
- categorical variables taking values in a finite set $\{C_1, \ldots, C_g\}$;
- binary variables in $\{0, 1\}$.

For target variables, the distinction between continuous and discrete values leads naturally to regression and classification tasks.

Most learning algorithms expect numerical input. A categorical variable with $k$ mutually exclusive categories can be represented by a one-hot vector

$$\boldsymbol{e}_i = (0, \ldots, 0, \underbrace{1}_{i\text{-th position}}, 0, \ldots, 0)^\top, \qquad i \in \{1, \ldots, k\}. \tag{1}$$

For categories with a natural order, an encoding should reflect that ordinality, for example through an ordered sequence of integers.

**Notation.** In equation (1), $k$ is the number of categories and $\boldsymbol{e}_i$ is the binary indicator for category $i$; exactly one coordinate equals 1.

### 9.3 Labels and dataset notation

Entries in the target column are called **labels**. Data are **labeled** when the target is observed and **unlabeled** when it is unknown.

Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ the output/target space, with $\mathcal{X} \subseteq \mathbb{R}^p$ in the typical vector-valued case. The dataset is written

$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, (\boldsymbol{x}^{(n)}, y^{(n)})\} \subset (\mathcal{X} \times \mathcal{Y})^n. \tag{2}$$

Each observation satisfies

$$(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{X} \times \mathcal{Y}, \qquad i = 1, \ldots, n, \tag{3}$$

and the $j$-th feature column can be represented as

$$\boldsymbol{x}_j = \left(x_j^{(1)}, \ldots, x_j^{(n)}\right)^\top \in \mathbb{R}^n. \tag{4}$$

**Notation.** $n$ is the number of observations; $p$ is the number of input features; $\boldsymbol{x}^{(i)}$ is the feature vector for observation $i$; $y^{(i)}$ is its target; and $\boldsymbol{x}_j$ collects feature $j$ across all $n$ observations.

## 10 Data Generation and the i.i.d. Assumption

The observed dataset $D$ is assumed to arise from an underlying joint probability distribution $P_{XY}$ defined on $\mathcal{X} \times \mathcal{Y}$. The true distribution is generally unknown. Supervised machine learning seeks to learn enough of its structure to predict $y$ from $\boldsymbol{x}$.

A standard assumption is that samples are drawn independently and identically distributed (i.i.d.) from $P_{XY}$. This contains two assumptions:

1. **Identically distributed:** every sample follows the same underlying distribution.
2. **Independent:** the realization of one observation does not depend on the other $n - 1$ observations.

This is a strong assumption and may fail in more complex settings such as time series.

## 11 Data Points as Vectors

For vector representations in the input space $\mathcal{X}$, assume that the dataset has been **mean-centered**: the mean along each input dimension is subtracted so that the data cloud is centered around the origin. This operation is reversible as long as the feature means are retained.

**Notation.** The dataset notation above uses $n$ observations and $p$ features. In the linear-algebra development below, $P$ denotes the number of data points and $N$ the ambient dimension. Thus $P$ plays the same counting role as $n$, while $N$ plays the same dimensional role as $p$.

Each observation can then be viewed as a point—equivalently, a vector—in a multidimensional vector space. A vector space is a collection of vectors closed under vector addition and scalar multiplication.

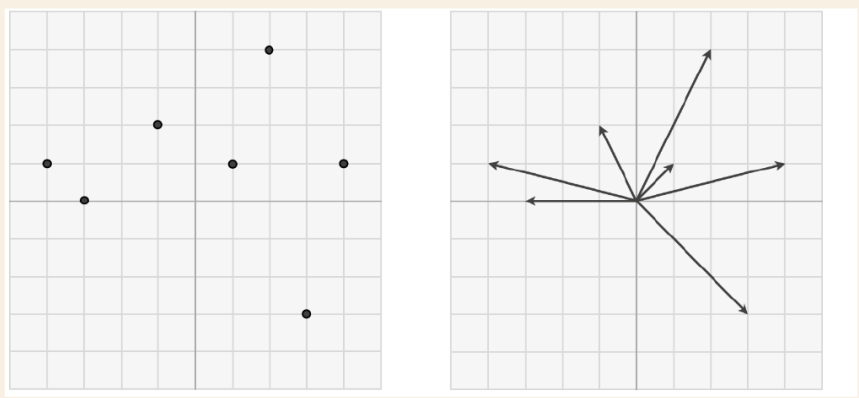


**Figure 21.** Two equivalent visualizations of the same 2-D data: points (left) and arrows from the origin (right).

# 12 Spanning Sets and Bases

## 12.1 Spanning sets

**Definition**

A **spanning set** is a collection of vectors whose linear combinations can produce every vector in the space being spanned.

In two dimensions, the standard vectors

$$\boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

span the plane because any point $\begin{bmatrix} x & y \end{bmatrix}^\top$ can be written as

$$\begin{bmatrix} x \\ y \end{bmatrix} = x\boldsymbol{e}_1 + y\boldsymbol{e}_2. \tag{5}$$

## 12.2 Basis

**Definition**

A **basis** is a linearly independent spanning set. In an $N$-dimensional vector space, a basis therefore contains exactly $N$ vectors. A more general spanning set may contain more than $N$ vectors, but then it is not a basis in the minimal sense.

For a candidate basis $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_N$ and each data point $\boldsymbol{x}_p$, perfect representation means that there are weights $w_{1,p}, \ldots, w_{N,p}$ such that

$$\sum_{n=1}^{N} \boldsymbol{c}_n w_{n,p} = \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{6}$$

**Notation.** $N$ is the ambient dimension; $P$ is the number of data points; $\boldsymbol{c}_n$ is basis vector $n$; $w_{n,p}$ is the coordinate/weight of data point $p$ along $\boldsymbol{c}_n$; $\boldsymbol{x}_p$ is the original point.

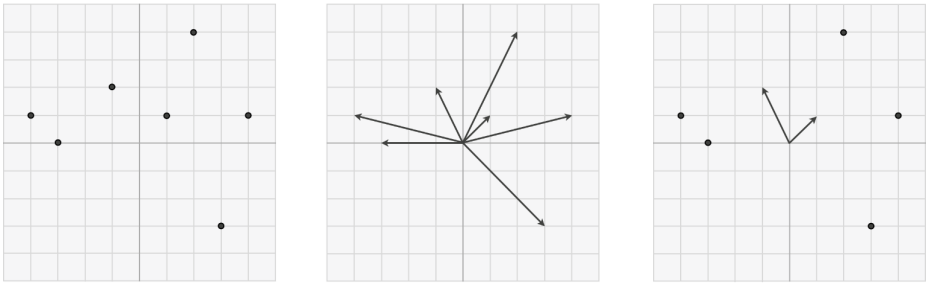


**Figure 22.** Points, vectors from the origin, and a basis representation in two dimensions.

## 13 Standard and Orthonormal Bases

### 13.1 The standard basis

The $k$-th standard basis vector has zeros everywhere except for a 1 in position $k$:

$$\boldsymbol{c}_k = (0, \ldots, 0, 1, 0, \ldots, 0)^\top.$$

For distinct basis vectors, the inner product is zero, while each vector has unit Euclidean norm:

$$\boldsymbol{c}_n^\top \boldsymbol{c}_m = 0, \qquad n \neq m, \tag{7}$$

$$\|\boldsymbol{c}_n\|_2^2 = 1, \qquad n = 1, \ldots, N. \tag{8}$$

Thus the standard basis is orthonormal. If

$$C = [\boldsymbol{c}_1 \; \boldsymbol{c}_2 \; \cdots \; \boldsymbol{c}_N],$$

then

$$C^\top C = I_{N \times N}. \tag{9}$$

In the standard basis, the coordinate vector of a point is simply the point itself: $\boldsymbol{w}_p = \boldsymbol{x}_p$.

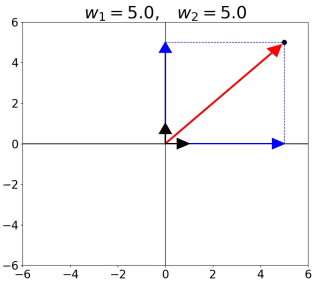



**Figure 23.** Reconstruction in the two-dimensional standard basis: a point $(5, 5)$ is reconstructed from weights $w_1 = 5$ and $w_2 = 5$.

### 13.2 Encoding with an orthonormal basis

For an orthonormal basis, the encoding of $\boldsymbol{x}_p$ is obtained directly by inner products:

$$\boldsymbol{w}_p = C^\top \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{10}$$

Reconstruction is exact when $C$ is a complete $N \times N$ orthonormal basis:

$$CC^\top \boldsymbol{x}_p = \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{11}$$

**Notation.** $C$ contains the orthonormal basis vectors as columns; $C^\top \boldsymbol{x}_p$ produces the coordinate vector $\boldsymbol{w}_p$; multiplying by $C$ decodes the coordinates back into the original data space.

## 14 Non-Standard Bases and Coordinate Transformations

For a non-standard basis, the coordinate vector generally differs numerically from the original data point. The encoding is

$$\boldsymbol{w}_p = \begin{bmatrix} w_{1,p} \\ w_{2,p} \\ \vdots \\ w_{N,p} \end{bmatrix}, \tag{12}$$

which represents $\boldsymbol{x}_p$ in a transformed feature space whose coordinate axes are defined by the spanning vectors.

Consider the basis vectors

$$\boldsymbol{c}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \boldsymbol{c}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}. \tag{13}$$

Because these vectors are linearly independent, any point in $\mathbb{R}^2$ can be expressed uniquely as

$$w_{1,p}\boldsymbol{c}_1 + w_{2,p}\boldsymbol{c}_2 = \boldsymbol{x}_p. \tag{14}$$

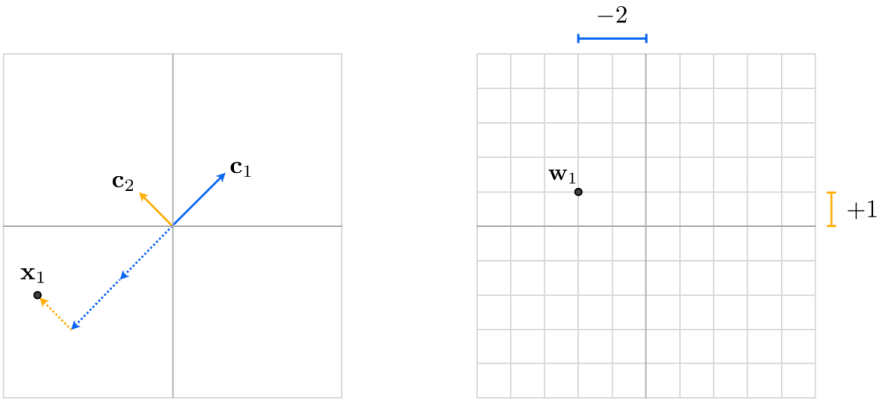



**Figure 24.** A point $\boldsymbol{x}_1$ in the original coordinates (left) and its encoding $\boldsymbol{w}_1$ in coordinates defined by the non-standard basis (right).

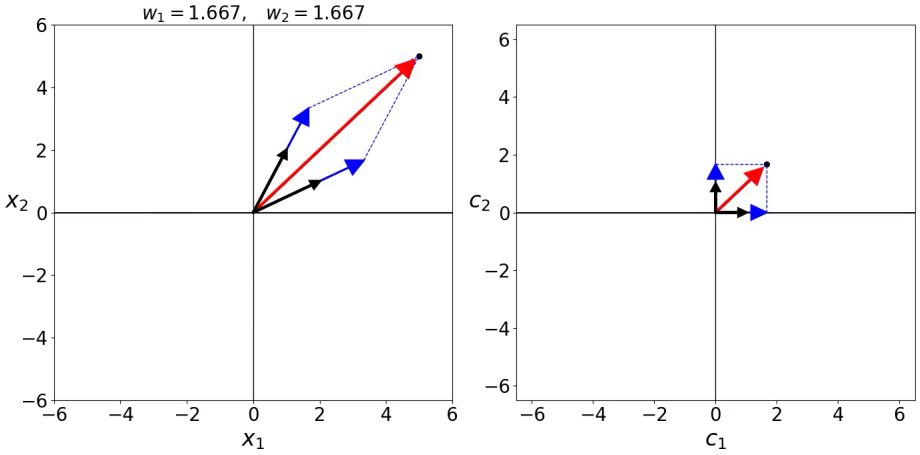



**Figure 25.** Geometric decomposition in a non-standard basis: the red vector is reconstructed from scaled basis vectors, while the coordinate vector records the required weights.

## 15 Finding the Encoding Weights

Let the basis/spanning vectors be stacked column-wise into

$$C = [\boldsymbol{c}_1 \; \boldsymbol{c}_2 \; \cdots \; \boldsymbol{c}_N], \tag{15}$$

and let $\boldsymbol{w}_p$ contain the weights for point $\boldsymbol{x}_p$. The squared reconstruction cost is

$$h(\boldsymbol{w}_p) = \|C\boldsymbol{w}_p - \boldsymbol{x}_p\|_2^2. \tag{16}$$

Setting the gradient to zero yields the symmetric first-order system

$$C^\top C\boldsymbol{w}_p = C^\top \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{17}$$

For all $P$ points together, the optimization problem is

$$\underset{\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_P}{\text{minimize}} \; \frac{1}{P} \sum_{p=1}^{P} \|C\boldsymbol{w}_p - \boldsymbol{x}_p\|_2^2. \tag{18}$$

**Notation.** $C$ is the matrix of spanning vectors; $\boldsymbol{w}_p$ is the coordinate/encoding vector; $\boldsymbol{x}_p$ is the original point; $h(\boldsymbol{w}_p)$ is squared reconstruction error. Equation (17) is the normal-equation form obtained from the first-order optimality condition.

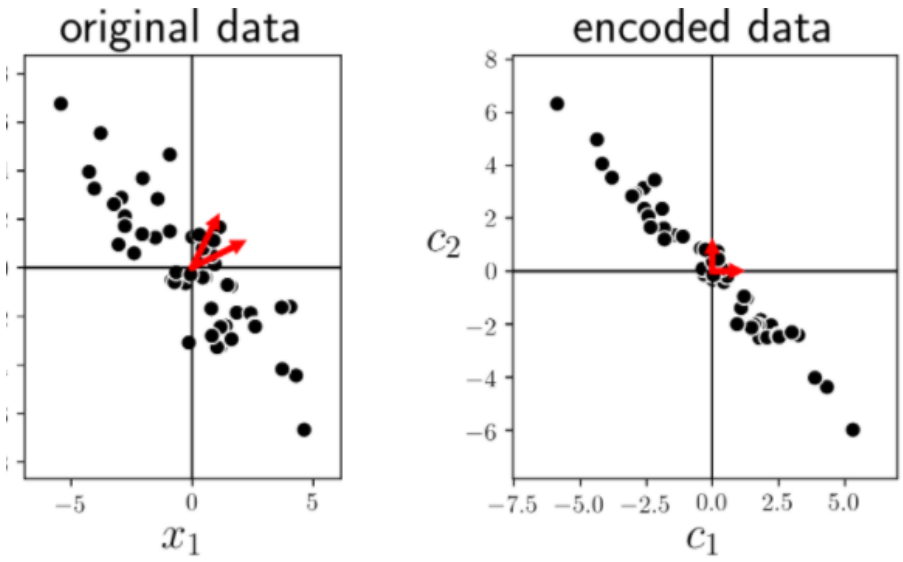



**Figure 26.** Original data versus the same observations represented in the coordinate system defined by a non-standard spanning set.

## 16 Subspaces and Projection

Up to this point, perfect representation in a generic $N$-dimensional space requires enough independent spanning directions. If only $K < N$ independent vectors are used, they can span at most a $K$-dimensional subspace of the full $N$-dimensional space. For example, in three dimensions, $K = 2$ independent vectors span a plane, while $K = 1$ spans a line.

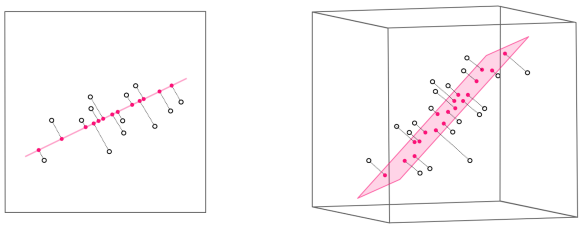


**Figure 27.** Lower-dimensional subspaces: a line or plane cannot represent every point in the ambient space exactly.

For $K$ spanning vectors, the matrix now has shape

$$C = [\boldsymbol{c}_1 \ \cdots \ \boldsymbol{c}_K] \in \mathbb{R}^{N \times K},$$

but the first-order system keeps the same form:

$$C^\top C \boldsymbol{w}_p = C^\top \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{19}$$

## 16.1 Encoding, decoding, and projection

When $K < N$, the point cannot in general be reconstructed exactly. Instead, encoding maps the point to $K$ coordinates, and decoding combines those coordinates with the $K$ basis vectors to obtain the closest representable point in the subspace. Geometrically, this is an orthogonal **projection**: dropping the point perpendicularly onto the subspace.

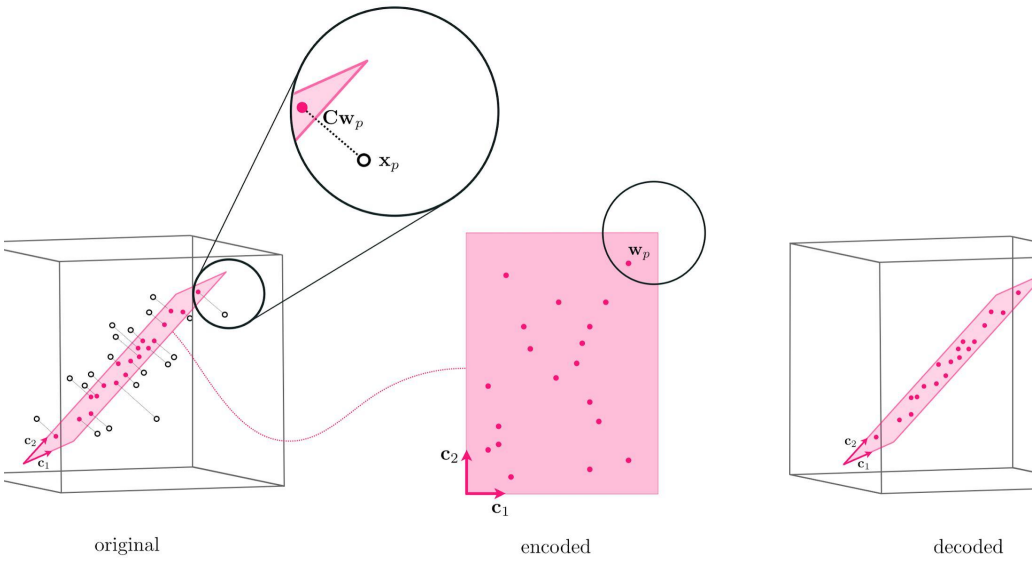



**Figure 28.** Encoding and decoding in a lower-dimensional subspace. The decoded point lies in the chosen lower-dimensional subspace and approximates the original point.

If the $K$ subspace basis vectors are orthonormal, the encoding again simplifies to

$$\boldsymbol{w}_p = C^\top \boldsymbol{x}_p, \qquad p = 1, \ldots, P, \tag{20}$$

and decoding gives the orthogonal projection

$$CC^\top \boldsymbol{x}_p \approx \boldsymbol{x}_p, \qquad p = 1, \ldots, P. \tag{21}$$

The approximation becomes equality only when the original point already lies in the spanned subspace (or when the orthonormal set is a complete basis of the whole space).

**Notation.** For an orthonormal $N \times K$ matrix $C$, $C^\top \boldsymbol{x}_p$ is the $K$-dimensional encoding and $CC^\top \boldsymbol{x}_p$ is the decoded projection in the original $N$-dimensional space. The residual $\|CC^\top \boldsymbol{x}_p - \boldsymbol{x}_p\|_2$ measures the projection/reconstruction error between the original point and its projection.

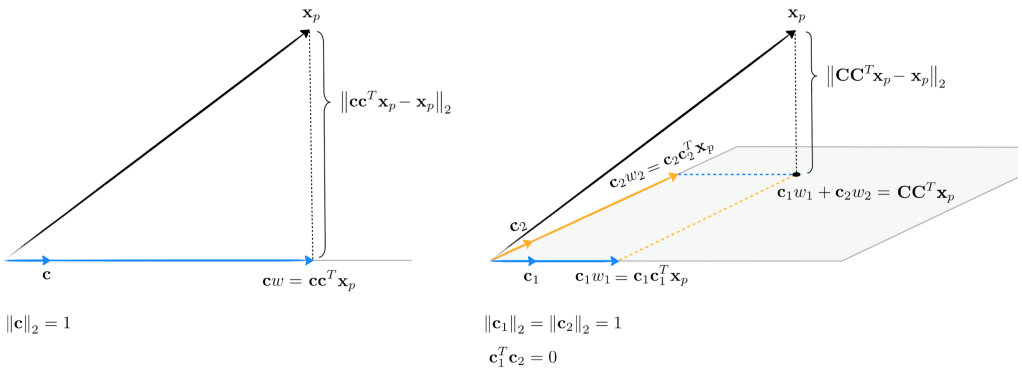



**Figure 29.** Geometric interpretation of orthonormal projection for one and two basis directions. The residual is perpendicular to the spanned subspace.

### Vector Spaces and Projections — Key Takeaways

- A supervised dataset pairs feature vectors with targets; categorical features often require numerical encoding.
- Samples are modeled as independent, identically distributed draws from an unknown joint distribution $P_{XY}$.
- A spanning set generates a space through linear combinations; a basis is a minimal linearly independent spanning set.
- Orthonormal bases make encoding especially simple: coordinates are inner products, $\boldsymbol{w} = C^\top \boldsymbol{x}$.
- A non-standard basis changes coordinates but can still represent the same point exactly when it spans the entire space.
- With only $K < N$ directions, decoding produces a projection onto a lower-dimensional subspace rather than an exact reconstruction in general.

Part III

# Optimization and Automatic Differentiation

## 17 Optimization Fundamentals

Optimization searches for a parameter vector $\boldsymbol{w}$ that minimizes or maximizes an **objective function** $g(\boldsymbol{w})$. When the objective is minimized in a learning problem, it is often called a **loss** or **cost function**. An optimizer is commonly denoted by $\boldsymbol{w}^\star$.

### 17.1 Global and local optima

A point $\boldsymbol{w}^\star$ is a global minimizer when

$$g(\boldsymbol{w}^\star) \le g(\boldsymbol{w}) \qquad \text{for every admissible } \boldsymbol{w}, \tag{22}$$

and a global maximizer when the inequality is reversed. Minimization and maximization are directly related:

$$\arg\max_{\boldsymbol{w}} g(\boldsymbol{w}) = \arg\min_{\boldsymbol{w}} [-g(\boldsymbol{w})].$$

A **local minimum** is smaller than or equal to the function values in a neighborhood of the point, but need not be the smallest value over the entire domain. Nonconvex objectives can therefore contain several local minima and maxima in addition to a global optimum.

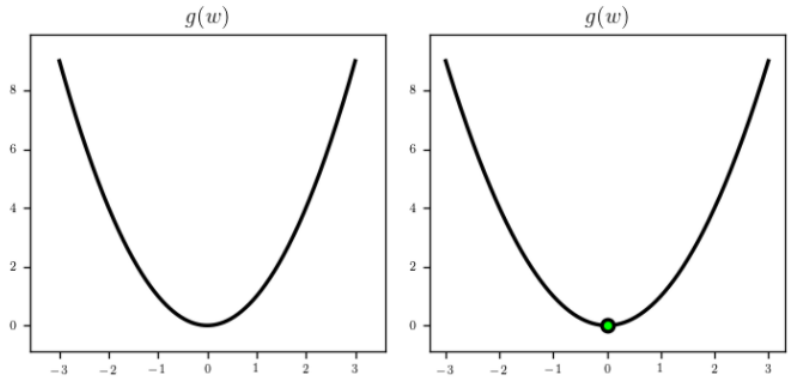



**Figure 30.** A convex quadratic has a unique global minimum. The highlighted point marks the minimizer of the objective.

### 17.2 Zero-order and first-order conditions

A zero-order definition compares objective values directly, as in equation (22). First-order conditions use derivatives. For a differentiable scalar function $g(w)$, every differentiable local optimum in the interior of the domain must satisfy

$$\frac{d}{dw} g(w^\star) = 0. \tag{23}$$

In $N$ dimensions the corresponding stationary-point condition is

$$\nabla g(\boldsymbol{w}^\star) = \mathbf{0}, \qquad \nabla g(\boldsymbol{w}) = \begin{bmatrix} \partial g/\partial w_1 \\ \partial g/\partial w_2 \\ \vdots \\ \partial g/\partial w_N \end{bmatrix}. \tag{24}$$

**Remark**

A zero gradient identifies a *stationary point*, not automatically a minimum. For example, $g(w) = w^3$ satisfies $g'(0) = 0$, yet $w = 0$ is neither a local minimum nor a local maximum. Additional information is needed to classify stationary points. Second-order information (the second derivative in one dimension or the Hessian in several dimensions) can help with this classification, although the optimization development below focuses mainly on first-order methods.

In simple problems, the stationary equations can sometimes be solved analytically in closed form. For more complicated objectives, a closed-form solution may be unavailable or algebraically impractical, which motivates iterative procedures such as gradient descent.

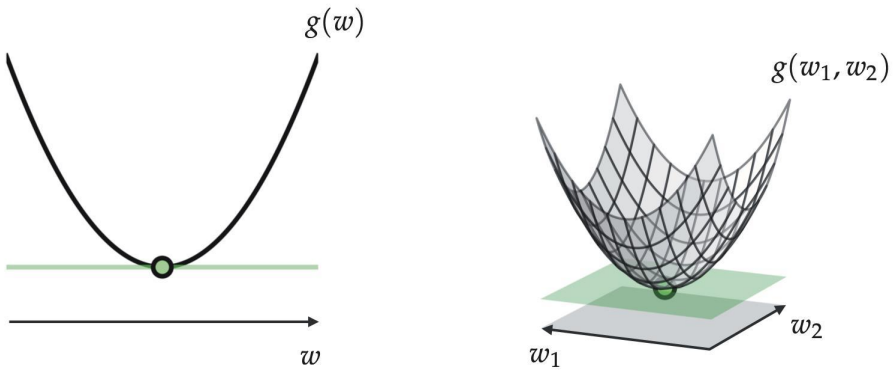



**Figure 31.** At a differentiable minimum, the first-order Taylor approximation is flat: a horizontal tangent in one dimension and a horizontal tangent hyperplane in higher dimensions.

## 18 Local Linear Approximation and the Gradient

For a differentiable function $g : \mathbb{R}^N \to \mathbb{R}$, the first-order Taylor approximation around $\boldsymbol{w}^0$ is

$$h(\boldsymbol{w}) = g(\boldsymbol{w}^0) + \nabla g(\boldsymbol{w}^0)^\top (\boldsymbol{w} - \boldsymbol{w}^0). \tag{25}$$

Equivalently, $h(\boldsymbol{w}) = a + \boldsymbol{b}^\top \boldsymbol{w}$, where

$$\boldsymbol{b} = \nabla g(\boldsymbol{w}^0), \qquad a = g(\boldsymbol{w}^0) - \nabla g(\boldsymbol{w}^0)^\top \boldsymbol{w}^0.$$

The vector $\boldsymbol{b}$ determines the orientation of the tangent hyperplane; the scalar $a$ shifts it so that it touches $g$ at $\boldsymbol{w}^0$.

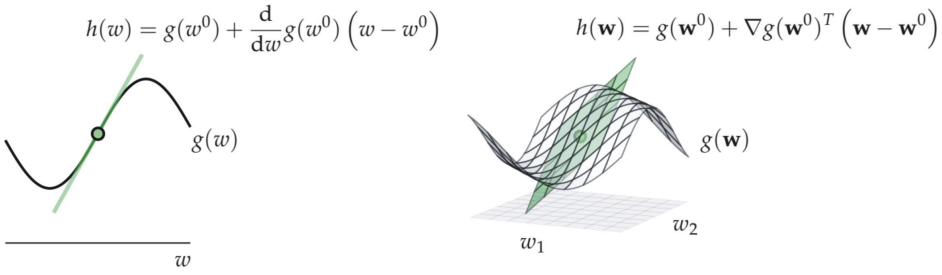



**Figure 32.** First-order Taylor approximation in one and two input dimensions. The tangent line or hyperplane matches the function locally at the expansion point.

## 18.1 Directions of steepest ascent and descent

Consider a linear function $h(\boldsymbol{w}) = a + \boldsymbol{b}^\top \boldsymbol{w}$ with $\boldsymbol{b} \neq \mathbf{0}$ and a unit direction $\boldsymbol{d}$, $\|\boldsymbol{d}\|_2 = 1$. With respect to the Euclidean norm, moving one unit from $\boldsymbol{w}^0$ changes the linear model by

$$h(\boldsymbol{w}^0 + \boldsymbol{d}) = a + \boldsymbol{b}^\top \boldsymbol{w}^0 + \boldsymbol{b}^\top \boldsymbol{d}.$$

Only $\boldsymbol{b}^\top \boldsymbol{d}$ depends on the direction. Since

$$\boldsymbol{b}^\top \boldsymbol{d} = \|\boldsymbol{b}\|_2 \|\boldsymbol{d}\|_2 \cos\theta = \|\boldsymbol{b}\|_2 \cos\theta,$$

the largest increase occurs when $\theta = 0$ and the largest decrease when $\theta = \pi$:

$$\boldsymbol{d}_{\text{asc}} = \frac{\boldsymbol{b}}{\|\boldsymbol{b}\|_2}, \qquad \boldsymbol{d}_{\text{desc}} = -\frac{\boldsymbol{b}}{\|\boldsymbol{b}\|_2}. \tag{26}$$

Applying this result to the local linear approximation, $\boldsymbol{b} = \nabla g(\boldsymbol{w}^0)$. Whenever $\nabla g(\boldsymbol{w}^0) \neq \mathbf{0}$, the normalized gradient gives the direction of steepest local ascent under the Euclidean norm, and its negative gives steepest local descent. At regular points, the gradient is normal to the corresponding smooth level curve or level surface.

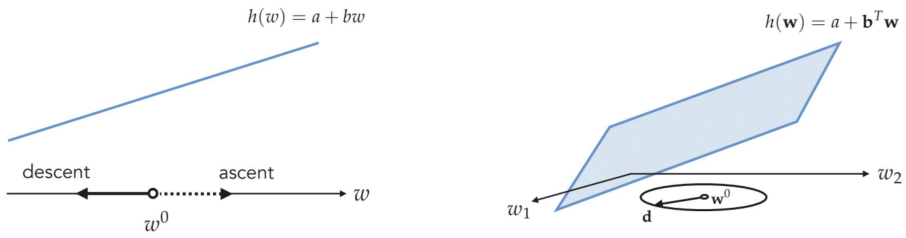



**Figure 33.** Ascent and descent directions for a linear function in one dimension and for a hyperplane in multiple dimensions.

**Gradient — Geometric Meaning**

At a point $\boldsymbol{w}^0$ with nonzero gradient, $\nabla g(\boldsymbol{w}^0)$ combines three pieces of local information: its direction gives steepest ascent under the Euclidean norm, its negative gives steepest descent, and its norm measures the first-order rate of increase in that steepest direction.

## 19 Computing Derivatives

Analytical differentiation can become cumbersome for deeply nested functions. A centered finite-difference approximation estimates a one-dimensional derivative at $w^0$ using a small $\epsilon$:

$$\left.\frac{dg}{dw}\right|_{w^0} \approx \frac{g(w^0+\epsilon) - g(w^0-\epsilon)}{2\epsilon}. \tag{27}$$

This is a numerical approximation, whereas automatic differentiation computes derivatives by systematically applying differentiation rules to the operations that define the function.

Two essential rules are

$$\frac{d}{dw} f(g(w)) = f'(g(w))\, g'(w), \qquad \text{(chain rule)} \tag{28}$$

$$\frac{d}{dw}[f(w)g(w)] = f'(w)g(w) + f(w)g'(w). \qquad \text{(product rule)} \tag{29}$$

### 19.1 Computation graphs

A computation graph decomposes a function into elementary operations. For

$$g(w) = \tanh(w)\cos(w) + \log(w), \qquad w > 0, \tag{30}$$

where the restriction $w > 0$ ensures that the logarithm is real-valued. One convenient decomposition is

$$a = \tanh(w), \qquad b = \cos(w), \qquad c = \log(w), \qquad d = ab, \qquad e = d + c = g(w).$$

Each node is a function of its parent nodes, and evaluation proceeds from the input toward the final output.

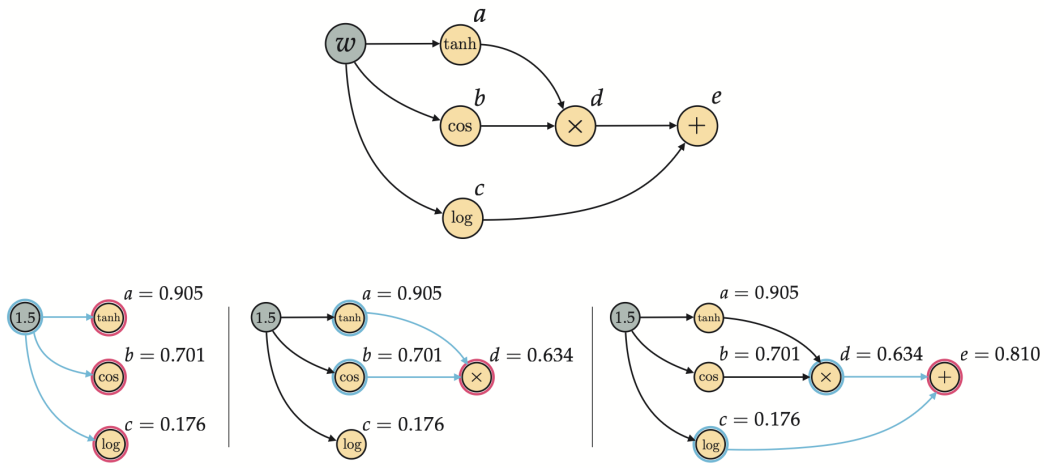



**Figure 34.** Computation graph for $g(w) = \tanh(w)\cos(w) + \log(w)$, together with a forward evaluation through its elementary operations.

## 19.2 Forward-mode automatic differentiation

Forward mode propagates both function values and derivatives from the input toward the output. Starting from $dw/dw = 1$,

$$\frac{da}{dw} = 1 - \tanh^2(w), \qquad \frac{db}{dw} = -\sin(w), \qquad \frac{dc}{dw} = \frac{1}{w}.$$

For $d = ab$, the product rule gives

$$\frac{dd}{dw} = \frac{da}{dw}b + a\frac{db}{dw} = (1 - \tanh^2 w)\cos w - \tanh w \sin w,$$

and therefore

$$\frac{dg}{dw} = (1 - \tanh^2 w)\cos w - \tanh w \sin w + \frac{1}{w}. \tag{31}$$

Forward mode is direct and exact up to floating-point arithmetic. More generally, one forward sweep propagates derivative information for one chosen input/tangent direction (a Jacobian–vector product). For a scalar-valued function of many independent inputs, constructing the entire gradient by forward mode generally requires one seed direction per input, so its cost grows with the input dimension.

## 19.3 Reverse mode and backpropagation

Reverse mode first evaluates the graph forward, then propagates derivative information backward from the scalar output toward the inputs. Along each path, local derivatives are multiplied by the chain rule; contributions from multiple child paths are summed.

For the two-input quadratic

$$g(w_1, w_2) = w_1^2 + w_2^2,$$

reverse accumulation recovers

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}.$$

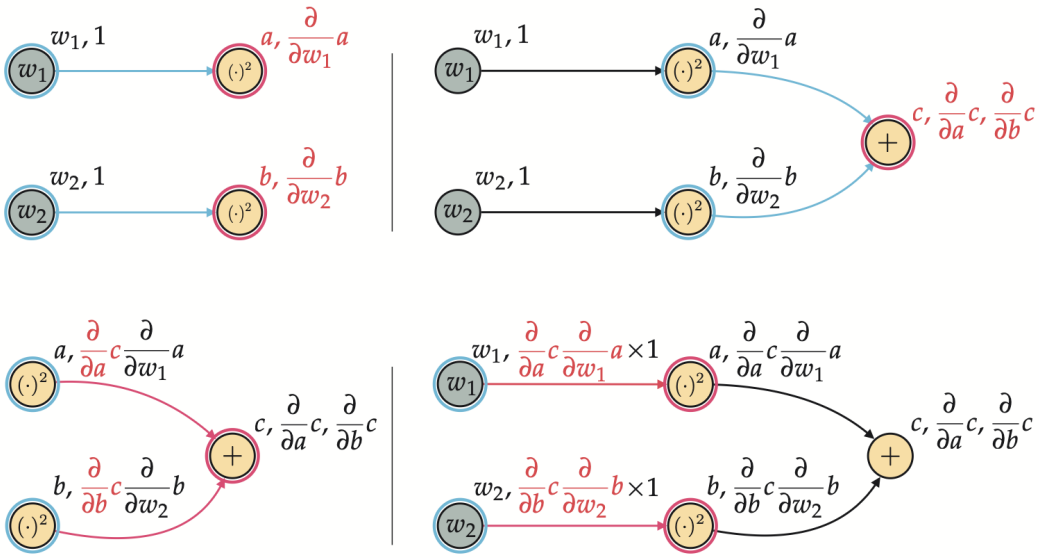



**Figure 35.** Reverse-mode differentiation on a two-input quadratic computation graph. Local derivatives are accumulated backward to obtain the gradient with respect to the inputs.

Reverse mode is particularly efficient for the common machine-learning setting of many parameters and one scalar loss: after the forward evaluation, one reverse sweep can accumulate the full gradient with respect to all inputs/parameters. It is the foundation of backpropagation and automatic-gradient systems used in modern deep-learning frameworks. Its main cost is that intermediate values from the forward pass must be retained for the backward sweep.

## 20 Gradient Descent

Gradient descent iteratively updates the parameters in the negative-gradient direction:

$$\boldsymbol{w}^{(k)} = \boldsymbol{w}^{(k-1)} - \alpha \nabla g\left(\boldsymbol{w}^{(k-1)}\right), \tag{32}$$

where $\alpha > 0$ is the **step size** or **learning rate**. When the gradient is nonzero, $-\nabla g$ is the steepest first-order descent direction under the Euclidean norm. A finite update decreases the objective only when the step size is appropriate for the local geometry and regularity of $g$.

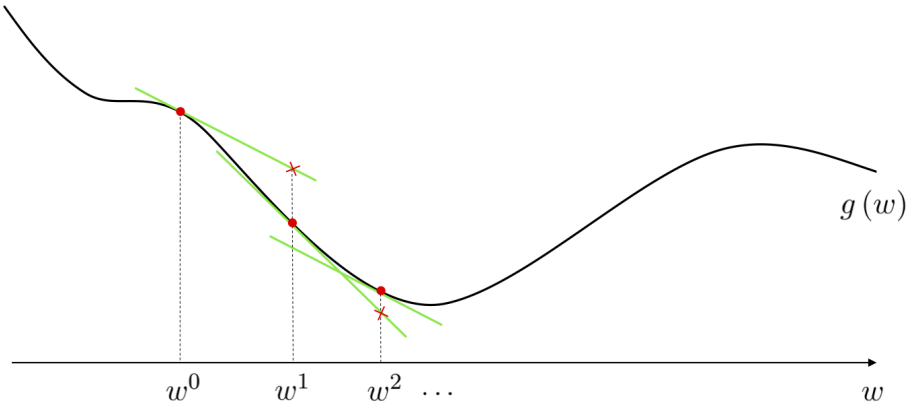



**Figure 36.** Gradient descent viewed through successive local tangent approximations. Each iterate moves in the negative-gradient direction before recomputing the local model.

## 20.1 Convex and nonconvex objectives

For a convex function, every local minimum is also a global minimum. For nonconvex objectives, different initializations can converge to different local minima or stall near stationary points. Consequently, both the initialization $\boldsymbol{w}^{(0)}$ and the step size $\alpha$ can materially affect the optimization trajectory.

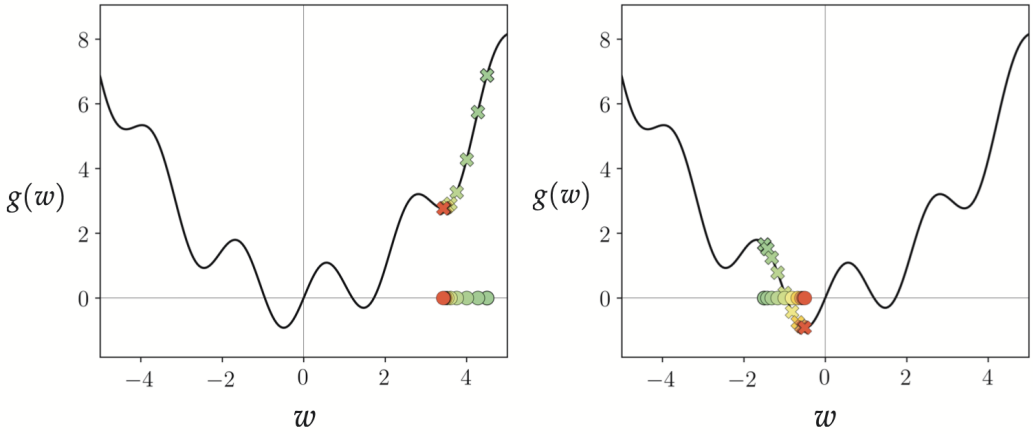



**Figure 37.** Gradient descent on a nonconvex one-dimensional objective from two different initializations. The trajectories can converge to different minima.

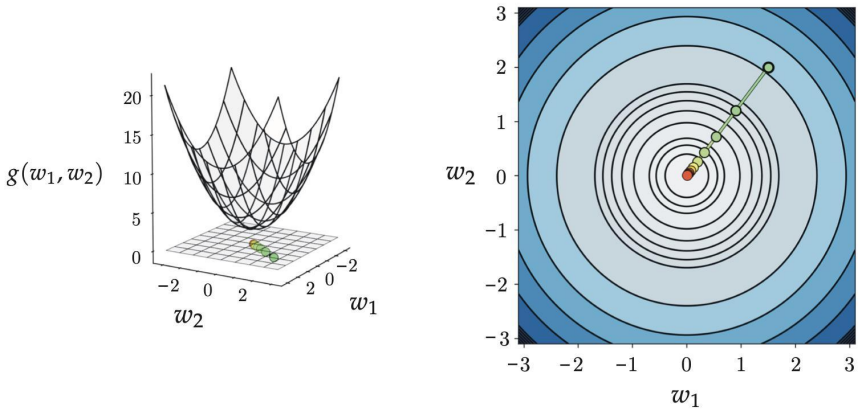



**Figure 38.** Gradient descent on a convex quadratic objective. The iterates move smoothly toward the unique global minimum.

## 20.2 Choosing the step size

The step size controls how far each update moves. A value that is too small produces slow progress; a value that is too large can overshoot the minimum and may even diverge. Illustrative schedules include a fixed step size, such as $\alpha = 10^{-m}$, and a diminishing schedule such as $\alpha_k = 1/k$ for $k \geq 1$. Whether a schedule converges depends on properties of the objective and on the step-size assumptions; no single schedule is universally convergent.

For the nonsmooth objective $g(w) = |w|$, the illustrated subgradient update with a fixed step oscillates around the minimizer, while the illustrated diminishing schedule progressively reduces the oscillation toward zero.

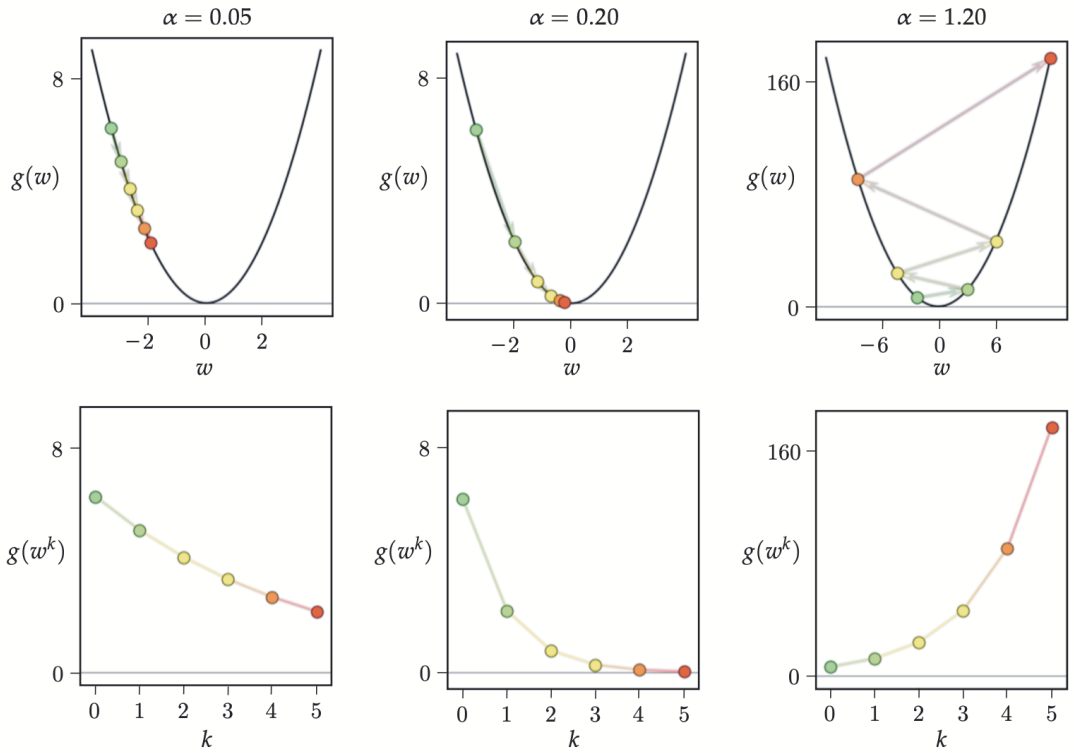



**Figure 39.** Effect of the step size on gradient descent for a quadratic objective: a small step converges slowly, a moderate step converges efficiently, and an excessively large step diverges.

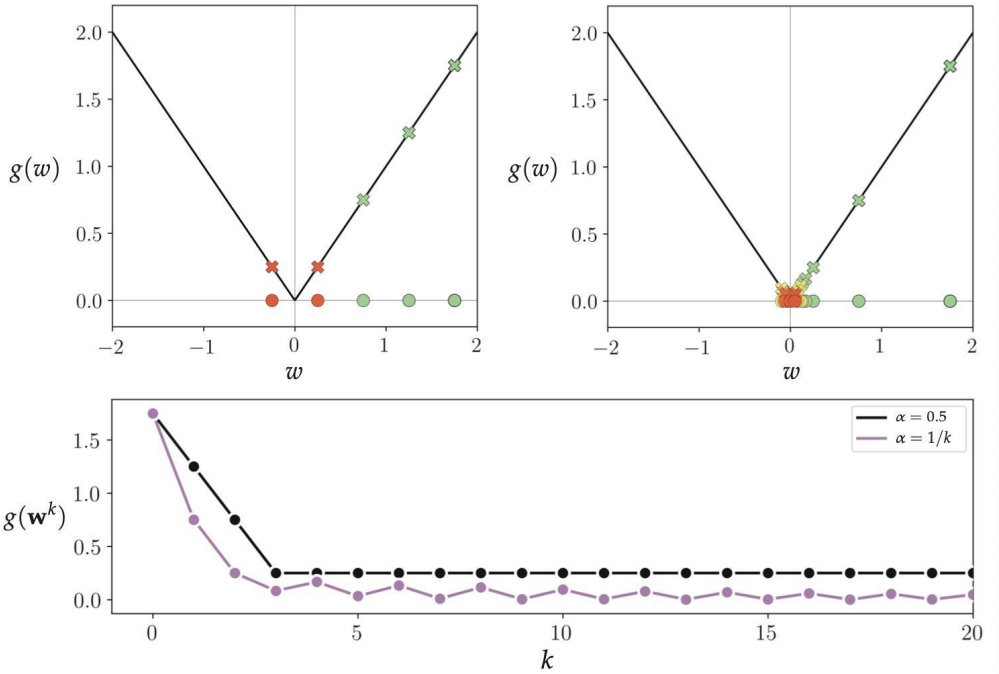



**Figure 40.** Fixed and diminishing step-length strategies for $g(w) = |w|$. The cost-history plot reveals persistent oscillation for the fixed step and convergence for the diminishing schedule.

## 20.3 Cost-function history plots

A cost-history plot displays $g(\boldsymbol{w}^{(k)})$ against iteration $k$. Such plots are useful for diagnosing divergence, oscillation, stagnation, and poorly chosen hyperparameters when the parameter space itself cannot be visualized directly. A useful trajectory need not decrease monotonically at every iteration; oscillatory behavior can still reach a lower final objective value.

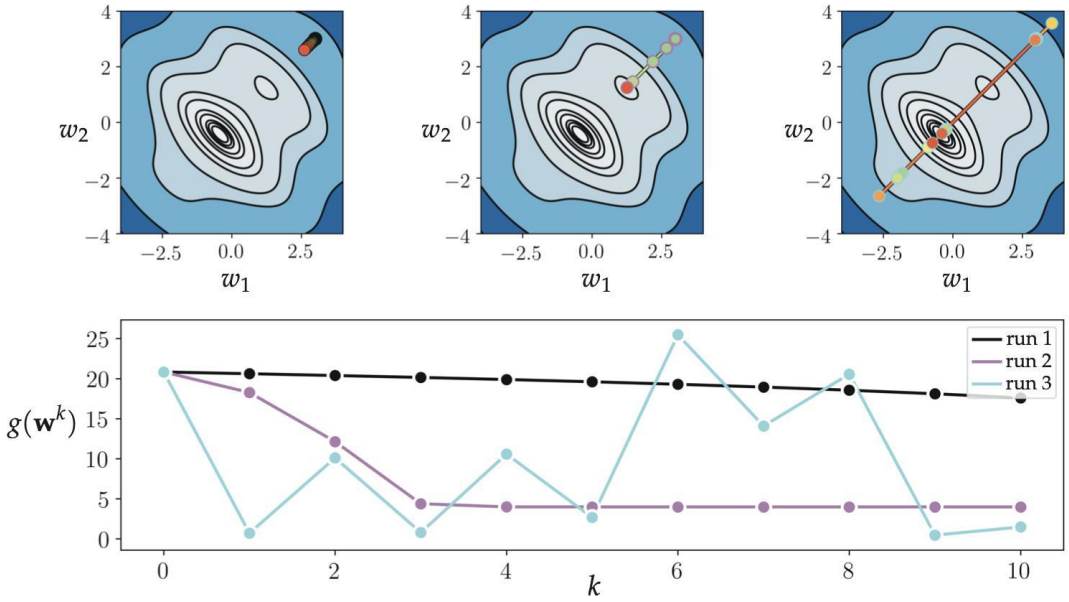



**Figure 41.** Three optimization trajectories on the same nonconvex objective and their cost histories. Different step sizes produce slow, locally trapped, or strongly oscillatory behavior.

## 20.4 Stopping criteria

Gradient descent is typically terminated when progress becomes negligible. Common criteria include

$$\|\nabla g(\boldsymbol{w}^{(k-1)})\|_2 \le \epsilon_g, \tag{33}$$

$$\|\boldsymbol{w}^{(k)} - \boldsymbol{w}^{(k-1)}\|_2 \le \epsilon_w, \tag{34}$$

$$\left|g(\boldsymbol{w}^{(k)}) - g(\boldsymbol{w}^{(k-1)})\right| \le \epsilon_f, \tag{35}$$

or a prescribed maximum number of iterations. Optional normalizations by the parameter dimension merely rescale the corresponding tolerance and should be stated explicitly when used. Near a stationary point, $\nabla g \approx \mathbf{0}$, so the update in equation (32) becomes very small.

## 20.5 Weaknesses of vanilla gradient descent

Two characteristic difficulties are associated with gradient direction and magnitude:

- **Zig-zagging.** In a narrow elongated valley, gradients are nearly perpendicular to closely spaced contours, so successive steps alternate across the valley and convergence slows.
- **Slow crawling.** Near minima or saddle-type stationary regions, the gradient norm can become very small, producing tiny updates even when a better region exists elsewhere.

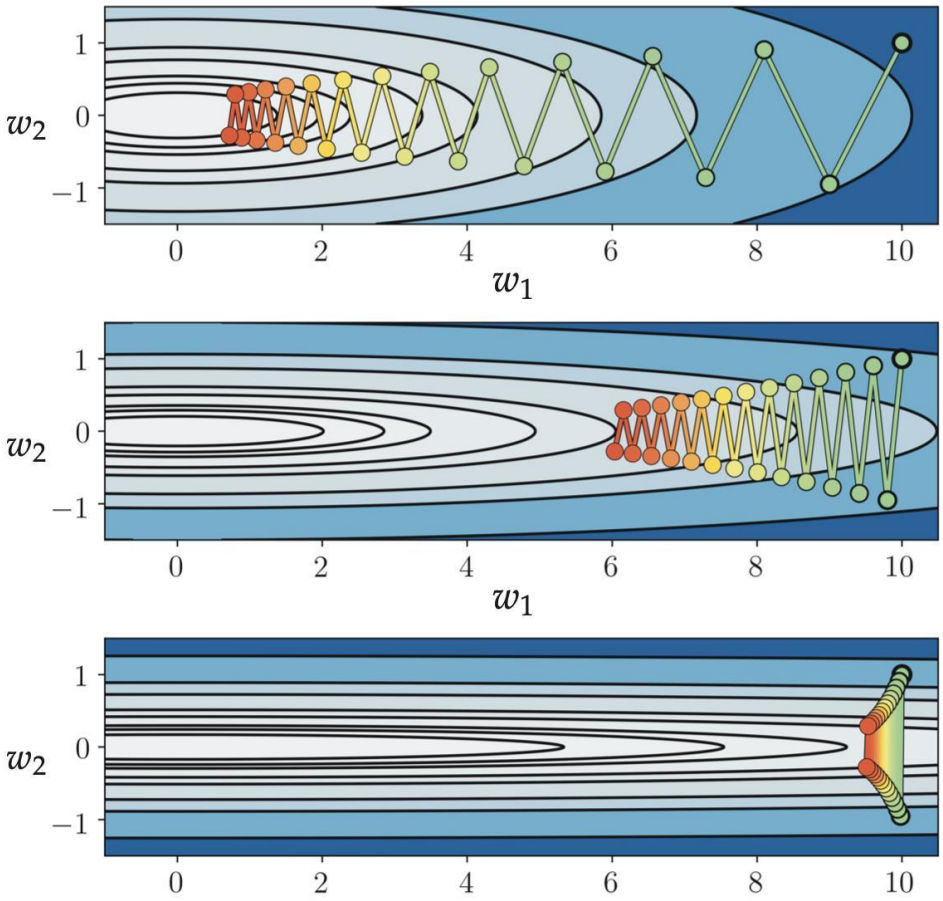



**Figure 42.** Increasing anisotropy in a quadratic objective produces progressively stronger zig-zagging under gradient descent.

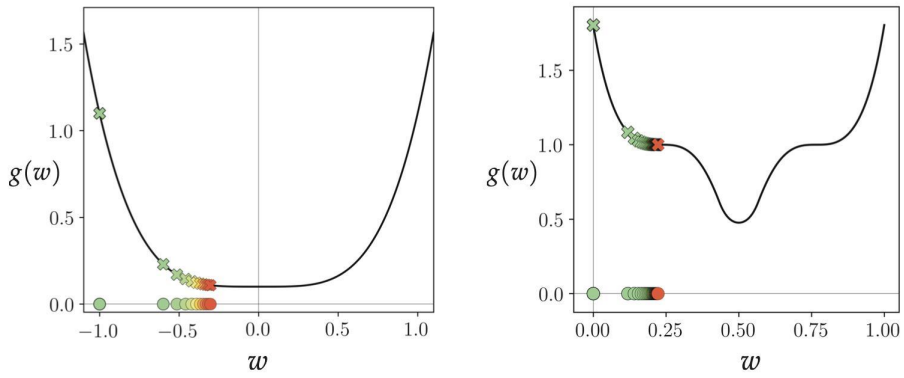



**Figure 43.** Small gradients cause slow progress near stationary regions; on a nonconvex objective, an iterate can effectively stall before reaching the best minimum.

## 20.6 Constrained gradient updates for PCA

The same optimization machinery can be applied to a matrix $C \in \mathbb{R}^{N\times K}$ whose columns are constrained to be orthonormal, $C^\top C = I_K$. Using the mean-centered data matrix $X \in \mathbb{R}^{N\times P}$, a variance-maximization formulation is

$$\underset{C}{\text{maximize}}\ \frac{1}{P}\operatorname{tr}(C^\top XX^\top C) = \operatorname{tr}(C^\top \Sigma C) \qquad \text{subject to} \qquad C^\top C = I_K, \tag{36}$$

where $\Sigma = \frac{1}{P}XX^\top$. A Euclidean gradient step for the equivalent minimization of the negative trace can be written, with the factor $2/P$ absorbed into the learning rate, as

$$C_k = C_{k-1} + \alpha XX^\top C_{k-1},$$

followed by re-orthonormalization so that $C_k^\top C_k = I_K$.

The equivalent reconstruction-error objective is

$$L(C) = \|CC^\top X - X\|_F^2. \tag{37}$$

Because $C$ occurs twice in $CC^\top$, the unrestricted Euclidean gradient contains two contributions. With

$$E(C) = CC^\top X - X,$$

matrix differentiation gives

$$\nabla_C L(C) = 2E(C)X^\top C + 2XE(C)^\top C. \tag{38}$$

When $C^\top C = I_K$ at the point where the gradient is evaluated,

$$E(C)^\top C = X^\top (CC^\top - I)C = \mathbf{0},$$

so equation (38) reduces to the simpler constrained expression

$$\nabla_C L(C) = 2(CC^\top X - X)X^\top C. \tag{39}$$

Thus the compact gradient used in the orthonormal PCA update is valid on the constraint set, not as the unrestricted gradient for arbitrary $C$. After an ordinary Euclidean step, re-orthonormalizing the columns restores the constraint. The next part derives why the constrained optimum is given by the principal-component directions.

**Optimization and Automatic Differentiation — Key Takeaways**

- First-order stationary points satisfy $\nabla g(\boldsymbol{w}) = \mathbf{0}$, but stationarity alone does not classify the point.
- The gradient is normal to level sets and gives the direction of steepest first-order ascent; $-\nabla g$ gives steepest descent.
- Computation graphs organize complex functions into elementary operations and make automatic differentiation systematic.
- Forward mode propagates derivatives from inputs to outputs; reverse mode propagates sensitivities from a scalar output back to many inputs.
- Gradient descent depends critically on step size, initialization, objective geometry, and stopping criteria.

Part IV

# Principal Component Analysis and Dimensionality Reduction

## 21 Linear Transformations and Projection

For $\boldsymbol{x} \in \mathbb{R}^n$ and $A \in \mathbb{R}^{m\times n}$, the matrix-vector product

$$\boldsymbol{y} = A\boldsymbol{x} \in \mathbb{R}^m$$

defines a linear transformation. Different matrix structures have different geometric effects: diagonal matrices scale coordinate axes; square orthonormal matrices rotate or reflect without changing Euclidean lengths; off-diagonal structure can shear; and rank-deficient maps can collapse data into a lower-dimensional subspace.

**Notation.** In the PCA sections, $X = [\boldsymbol{x}_1 \cdots \boldsymbol{x}_P] \in \mathbb{R}^{N\times P}$ stores $P$ mean-centered samples as columns; $C \in \mathbb{R}^{N\times K}$ contains $K$ retained directions; $C^\top C = I_K$; and $\boldsymbol{z}_p = C^\top \boldsymbol{x}_p \in \mathbb{R}^K$ is the encoded score vector.

For an orthonormal-column matrix $C \in \mathbb{R}^{N\times K}$, $C^\top C = I_K$. Encoding a point and decoding it are

$$\boldsymbol{w}_p = C^\top \boldsymbol{x}_p, \qquad C\boldsymbol{w}_p = CC^\top \boldsymbol{x}_p. \tag{40}$$

If $C$ is a complete $N \times N$ orthonormal basis, $CC^\top \boldsymbol{x}_p = \boldsymbol{x}_p$. If $C \in \mathbb{R}^{N\times K}$ with $K < N$, the decoded vector is the orthogonal projection onto the $K$-dimensional subspace and generally differs from the original point.

## 22 From a Fixed Basis to a Learned Basis

With a fixed basis $C$, the encoding weights minimize the reconstruction error

$$h(\boldsymbol{w}_p) = \|C\boldsymbol{w}_p - \boldsymbol{x}_p\|_2^2.$$

Principal Component Analysis (PCA) extends this idea by learning the spanning directions themselves. The joint linear encoding-decoding objective can be written as

$$g(\boldsymbol{w}_1, \ldots, \boldsymbol{w}_P, C) = \frac{1}{P}\sum_{p=1}^{P} \|C\boldsymbol{w}_p - \boldsymbol{x}_p\|_2^2. \tag{41}$$

For an orthonormal $C$, the optimal encoding is $\boldsymbol{w}_p = C^\top \boldsymbol{x}_p$, yielding

$$g(C) = \frac{1}{P}\sum_{p=1}^{P} \|CC^\top \boldsymbol{x}_p - \boldsymbol{x}_p\|_2^2. \tag{42}$$

This is a linear autoencoder viewpoint: $C^\top$ encodes each sample and $C$ decodes it back into the original space.

### 22.1 Reconstruction error and variance

For one mean-centered sample and orthonormal $C$,

$$\|CC^\top \boldsymbol{x}_p - \boldsymbol{x}_p\|_2^2 = \boldsymbol{x}_p^\top CC^\top CC^\top \boldsymbol{x}_p - 2\boldsymbol{x}_p^\top CC^\top \boldsymbol{x}_p + \boldsymbol{x}_p^\top \boldsymbol{x}_p \tag{43}$$

$$= -\|C^\top \boldsymbol{x}_p\|_2^2 + \|\boldsymbol{x}_p\|_2^2. \tag{44}$$

Because $\|\boldsymbol{x}_p\|_2^2$ does not depend on $C$, minimizing reconstruction error is equivalent to maximizing the squared norm of the projected coordinates.

If $C = [\boldsymbol{c}_1 \ \cdots \ \boldsymbol{c}_K]$, then

$$\|C^\top \boldsymbol{x}_p\|_2^2 = \sum_{n=1}^{K} (\boldsymbol{c}_n^\top \boldsymbol{x}_p)^2.$$

Thus learning an orthonormal basis for the retained $K$-dimensional subspace is equivalent to maximizing projected variance along $K$ mutually orthogonal directions.

## 23 Covariance and the First Principal Component

For a unit vector $\boldsymbol{c}_1$, define

$$h(\boldsymbol{c}_1) = \frac{1}{P} \sum_{p=1}^{P} (\boldsymbol{c}_1^\top \boldsymbol{x}_p)^2, \qquad \|\boldsymbol{c}_1\|_2 = 1. \tag{45}$$

When the data are mean-centered, this quantity is the **empirical variance under the $1/P$ normalization** along direction $\boldsymbol{c}_1$.

Stack the samples as

$$X = [\boldsymbol{x}_1 \ \boldsymbol{x}_2 \ \cdots \ \boldsymbol{x}_P] \in \mathbb{R}^{N \times P}.$$

Then

$$h(\boldsymbol{c}_1) = \frac{1}{P} \boldsymbol{c}_1^\top XX^\top \boldsymbol{c}_1 = \boldsymbol{c}_1^\top \Sigma \boldsymbol{c}_1, \qquad \Sigma = \frac{1}{P} XX^\top. \tag{46}$$

The matrix $\Sigma$ is the empirical covariance matrix under the $1/P$ normalization. Its diagonal entries are empirical feature variances and its off-diagonal entries are empirical covariances between features. The alternative factor $1/(P-1)$ gives the usual unbiased sample-covariance estimator; changing between these two global normalizations rescales the eigenvalues but does not change the PCA eigenvector directions.

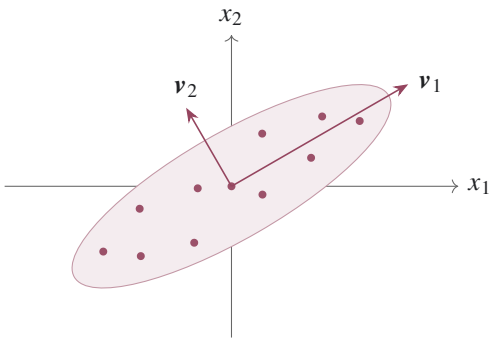



**Figure 44.** Geometric interpretation of a covariance matrix: orthogonal eigenvectors define the principal axes of an elongated data cloud, while the associated eigenvalues quantify spread along those axes.

## 24 Eigenvalues, Eigenvectors, and Principal Components

Because $\Sigma$ is real and symmetric, it admits an orthonormal eigendecomposition

$$\Sigma \boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i, \qquad \Sigma = V \Lambda V^\top, \qquad V^\top V = I. \tag{47}$$

For a covariance matrix, the eigenvalues are real and nonnegative. Ordering them as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0,$$

the Rayleigh-quotient objective in equation (46) is maximized by any unit eigenvector associated with the largest eigenvalue. Choosing one such vector as $\boldsymbol{c}_1 = \boldsymbol{v}_1$ gives a **first principal-component direction**, and $\lambda_1$ is the empirical variance captured along that direction.

If an eigenvalue is repeated, individual eigenvectors inside the corresponding eigenspace are not unique; an orthonormal basis of that eigenspace can be chosen without changing the PCA subspace.

The second direction maximizes

$$h(\boldsymbol{c}_2) = \boldsymbol{c}_2^\top \Sigma \boldsymbol{c}_2$$

subject to $\|\boldsymbol{c}_2\|_2 = 1$ and $\boldsymbol{c}_1^\top \boldsymbol{c}_2 = 0$. Its solution is $\boldsymbol{c}_2 = \boldsymbol{v}_2$. Repeating this construction gives the ordered principal directions.

For a $K$-dimensional representation, PCA selects

$$C_K = [\boldsymbol{v}_1 \; \boldsymbol{v}_2 \; \cdots \; \boldsymbol{v}_K], \tag{48}$$

so that encoding and reconstruction become

$$\boldsymbol{z}_p = C_K^\top \boldsymbol{x}_p, \qquad \hat{\boldsymbol{x}}_p = C_K \boldsymbol{z}_p = C_K C_K^\top \boldsymbol{x}_p. \tag{49}$$

**PCA — Core Equivalence**

For mean-centered data, minimizing linear reconstruction error over an orthonormal $K$-dimensional basis is equivalent to maximizing the variance captured by the projected coordinates. The global solution is given by the top $K$ eigenvectors of the covariance matrix.

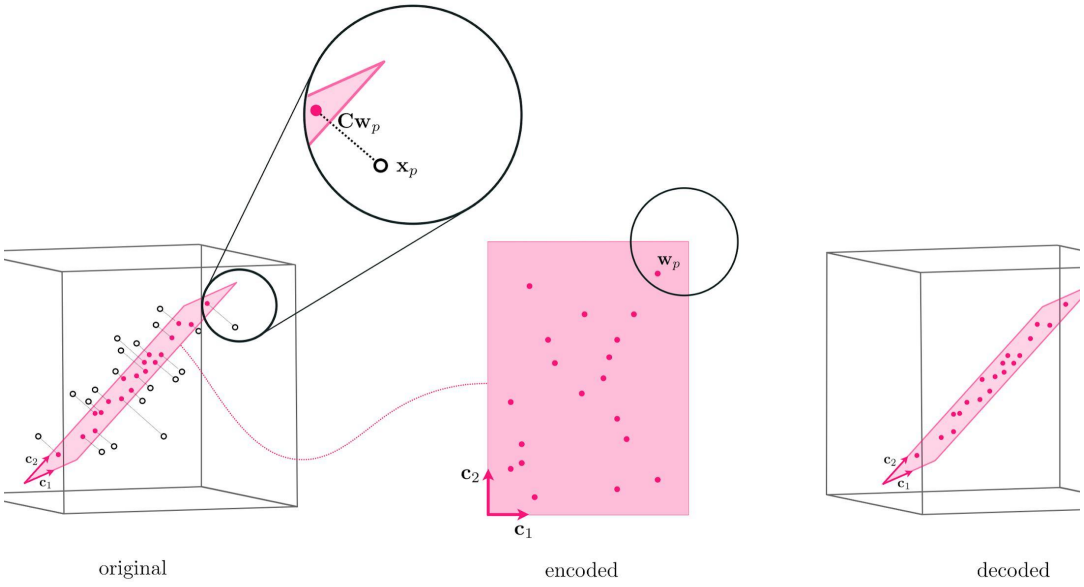



**Figure 45.** PCA with one retained component: original two-dimensional data are encoded to one coordinate and decoded onto the learned principal line.

## 25 Dimensionality Reduction and Unsupervised Representation Learning

PCA is unsupervised: no target labels are required. It learns orthogonal directions that explain the strongest correlations and variability in the input distribution. Keeping only $K < N$ principal components maps the data into a lower-dimensional representation while retaining as much variance as possible within a linear subspace.

The transformed coordinates are uncorrelated with respect to the empirical covariance model, and the representation can expose redundancy and dominant directions of variation. In downstream workflows, these coordinates can be used as engineered features or as a compressed representation of the original data.

A common summary of retained information is the **explained-variance ratio**. For the first $K$ principal directions,

$$\mathrm{EVR}(K) = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{N} \lambda_i}, \qquad \mathrm{EVR}(1) = \frac{\lambda_1}{\sum_{i=1}^{N} \lambda_i}. \tag{50}$$

### 25.1 A compact PCA exercise

**Exercise**

Consider the two-dimensional samples

$$\boldsymbol{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \boldsymbol{x}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{x}_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Let $\bar{\boldsymbol{x}} = \frac{1}{3}\sum_{p=1}^{3} \boldsymbol{x}_p$ and denote the centered samples by $\tilde{\boldsymbol{x}}_p = \boldsymbol{x}_p - \bar{\boldsymbol{x}}$. Compute, in order: the sample mean and the centered samples; the empirical covariance matrix $\Sigma = \frac{1}{3}\sum_{p=1}^{3} \tilde{\boldsymbol{x}}_p \tilde{\boldsymbol{x}}_p^{\top}$; the eigenvalues and eigenvectors of $\Sigma$; the first and second principal directions; the one-dimensional principal-component scores; the one-dimensional reconstructions and their mean-squared reconstruction

error; the explained-variance ratio of the first component; and the interpretation of the sign of the off-diagonal covariance entry.

## 26 Linear Structure, Manifolds, and Charts

Linear dimensionality reduction works well when high-dimensional data lie close to a low-dimensional *linear* subspace. More general datasets may instead lie near a curved low-dimensional structure.

A **manifold** is a topological space that locally resembles Euclidean space. For a smooth one-dimensional manifold, a local **chart** is a coordinate map $\phi : U \to V \subset \mathbb{R}$ from a suitable region $U$ of the manifold to an open Euclidean set $V$; its inverse $\phi^{-1} : V \to U$ is a local parameterization.

For example, on the open upper semicircle

$$U = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1,\ y > 0\},$$

the map $\phi(x, y) = x$ is one-to-one onto $(-1, 1)$, with inverse parameterization

$$\phi^{-1}(t) = \left(t, \sqrt{1 - t^2}\right), \qquad t \in (-1, 1).$$

The restriction to a local/open arc is essential: a single coordinate such as $x$ is not globally one-to-one on the entire circle.

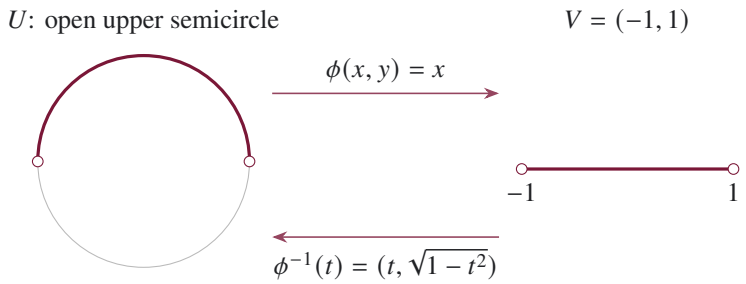



**Figure 46.** Local chart on the upper semicircle: $\phi$ maps a curved one-dimensional region to the Euclidean interval $(-1, 1)$, while $\phi^{-1}$ parameterizes the arc.

### Principal Component Analysis — Key Takeaways

- PCA learns orthonormal principal directions; the retained vectors form a basis of the selected principal subspace (and a full basis of the ambient space only when $K = N$).
- For mean-centered data, reconstruction-error minimization is equivalent to variance maximization.
- The covariance eigenvectors give principal directions and the eigenvalues quantify variance along those directions.
- Retaining only the leading $K$ eigenvectors gives a $K$-dimensional linear representation and an orthogonal reconstruction $C_K C_K^\top \boldsymbol{x}$.
- PCA is a linear method; curved low-dimensional structure motivates the broader language of manifolds and local charts.

## 27 Quick Review and Terminology

**Table 5.** Core concepts in machine learning, data representation, and linear algebra.

| Term | Meaning |
|---|---|
| Feature | Input property describing an observation |
| Target / label | Output variable to be predicted; a label is an observed target entry |
| One-hot encoding | $k$-dimensional binary representation with exactly one active coordinate |
| i.i.d. | Independent and identically distributed sampling assumption |
| Spanning set | Vectors whose linear combinations generate a space/subspace |
| Basis | Linearly independent spanning set; minimal representation of a vector space |
| Orthonormal basis | Basis vectors are mutually orthogonal and have unit norm |
| Encoding | Coordinate vector $\boldsymbol{w}$ representing a point relative to chosen spanning vectors |
| Decoding | Reconstruction $C\boldsymbol{w}$ in the original data space |
| Projection | Closest representable point in a lower-dimensional subspace, illustrated by $CC^\top \boldsymbol{x}$ for an orthonormal subspace basis |
| Supervised learning | Learning a mapping from labeled input/output examples |
| Unsupervised learning | Learning structure from data without target labels |
| Reinforcement learning | Learning a policy through state/action/reward interaction |
| Gradient | Vector of partial derivatives; at nonzero gradient, its direction gives steepest local ascent under the Euclidean norm |
| Automatic differentiation | Derivative evaluation by applying differentiation rules through a computation graph (exact up to floating-point arithmetic) |
| Reverse mode / backpropagation | Backward accumulation of sensitivities from a scalar output toward many inputs |
| Gradient descent | Iterative update $\boldsymbol{w}^{(k)} = \boldsymbol{w}^{(k-1)} - \alpha \nabla g(\boldsymbol{w}^{(k-1)})$ |
| Covariance matrix | Empirical matrix $\Sigma = \frac{1}{P} XX^\top$ describing variances and covariances of mean-centered data under the $1/P$ normalization |
| Principal-component direction | Covariance eigenvector defining a direction of maximal remaining variance |
| Principal-component score | Coordinate obtained by projecting a sample onto a principal direction, e.g. $z_i = \boldsymbol{v}_i^\top \boldsymbol{x}$ |
| PCA | Unsupervised linear dimensionality reduction using the leading covariance eigenvectors |

## References

[1] Deisenroth, Faisal, and Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] Simon Prince. *Understanding Deep Learning*. MIT Press, 2023.

[3] Watt, Borhani, and Katsaggelos. *Machine Learning Refined*, Chapters 2, 3, and 8, and Appendix B.

[4] Caron et al. (2018). *DeepCluster: Deep Clustering for Unsupervised Learning of Visual Features*.

[5] 3Blue1Brown. *Essence of Linear Algebra*, video series.